

DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Construyendo Agentes de Inteligencia Artificial

Día 3: Agentes - Frameworks, ADK

Miércoles 29 de Julio del 2026

Repaso Clase 2



**Agentes que me permiten
escribir código**

**y hacer subagentes que me
ayuden a escribir código**

**(de cualquier tipo: java, c++,
go, html, python, incluso
Agentes en LangChain o ADK)**



Ejemplos de Agentes

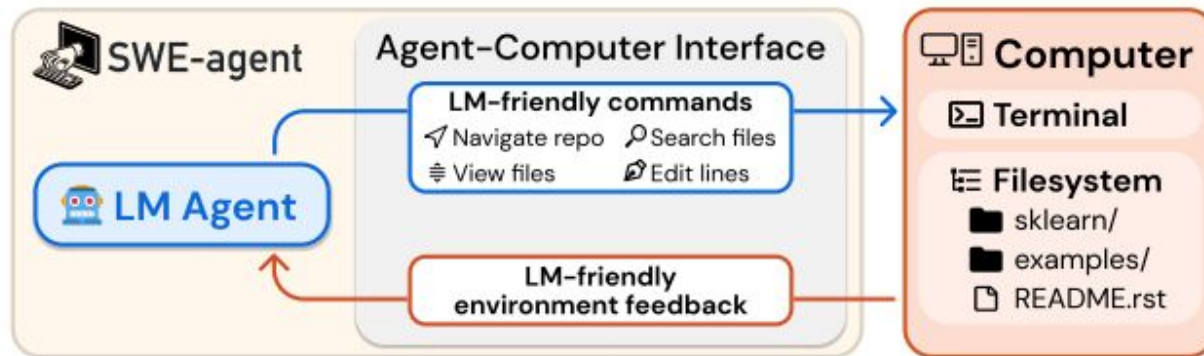
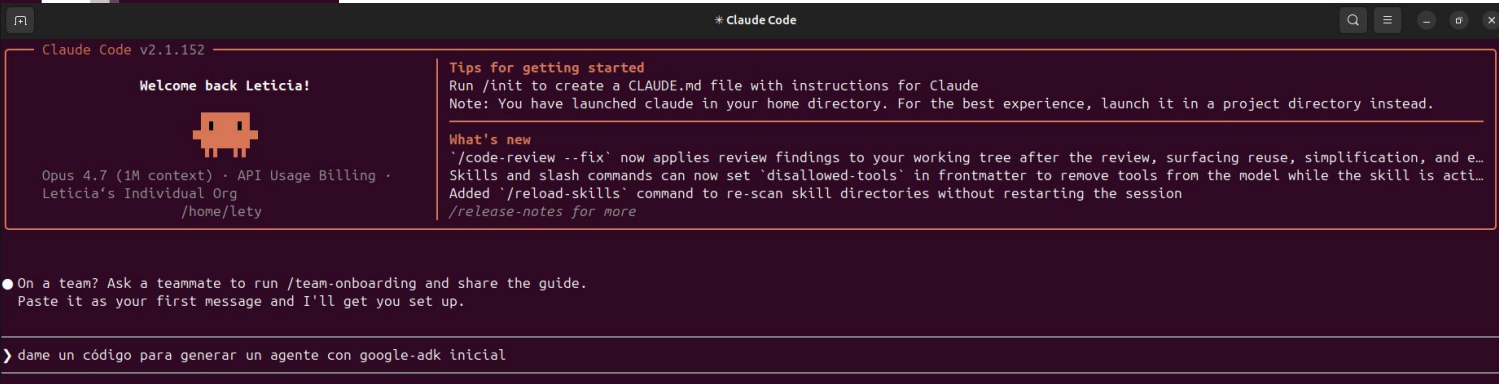
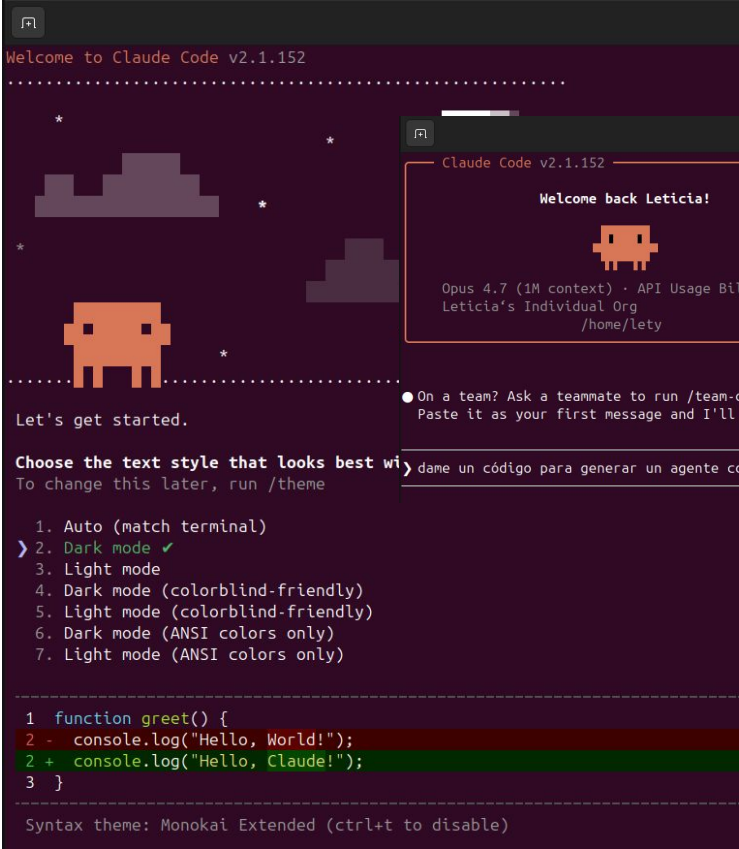


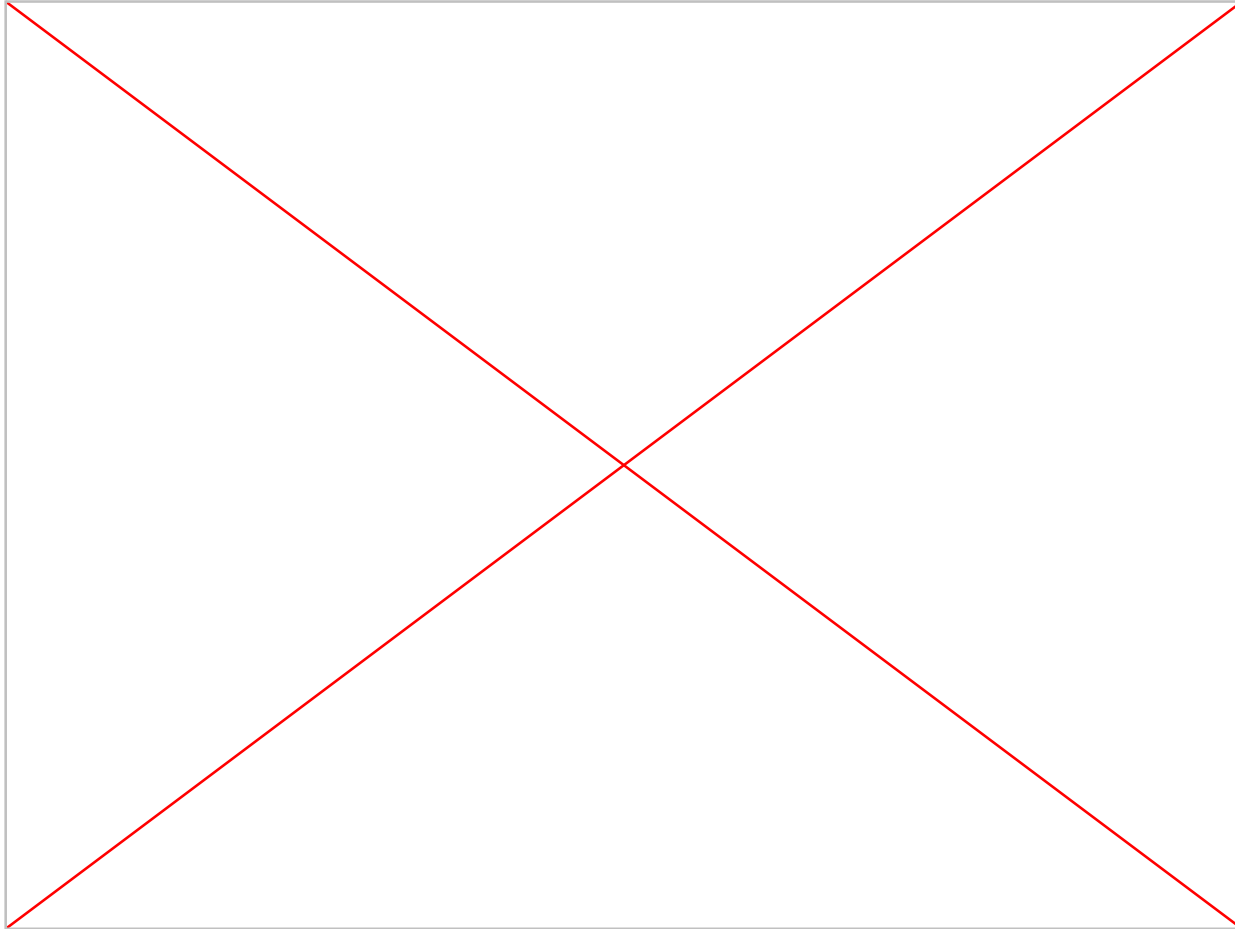
Figure 1: SWE-agent is an LM interacting with a computer through an agent-computer interface (ACI), which includes the commands the agent uses and the format of the feedback from the computer.

SWE-agent <https://arxiv.org/pdf/2405.15793>

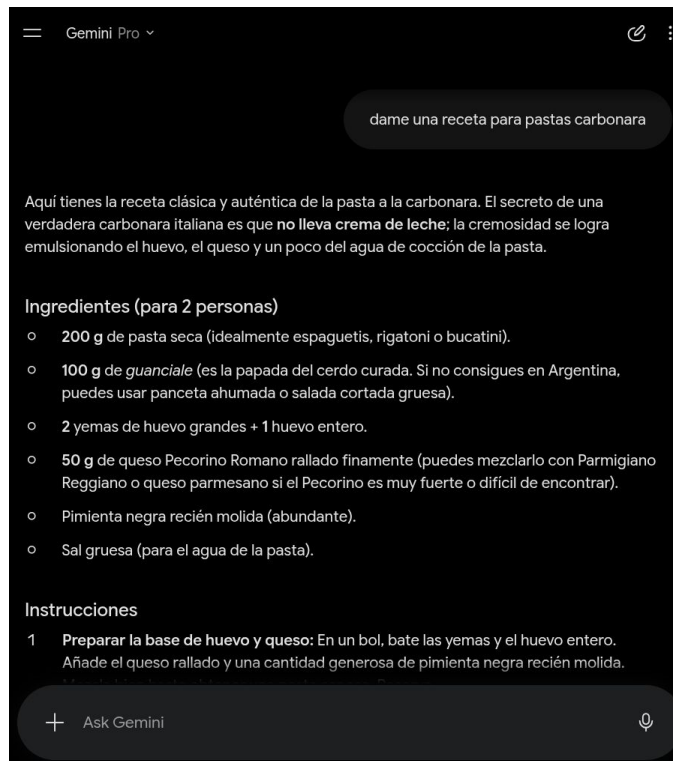
Ejemplos de Agentes: Claude Code



Docs: <https://code.claude.com/docs/en/overview>

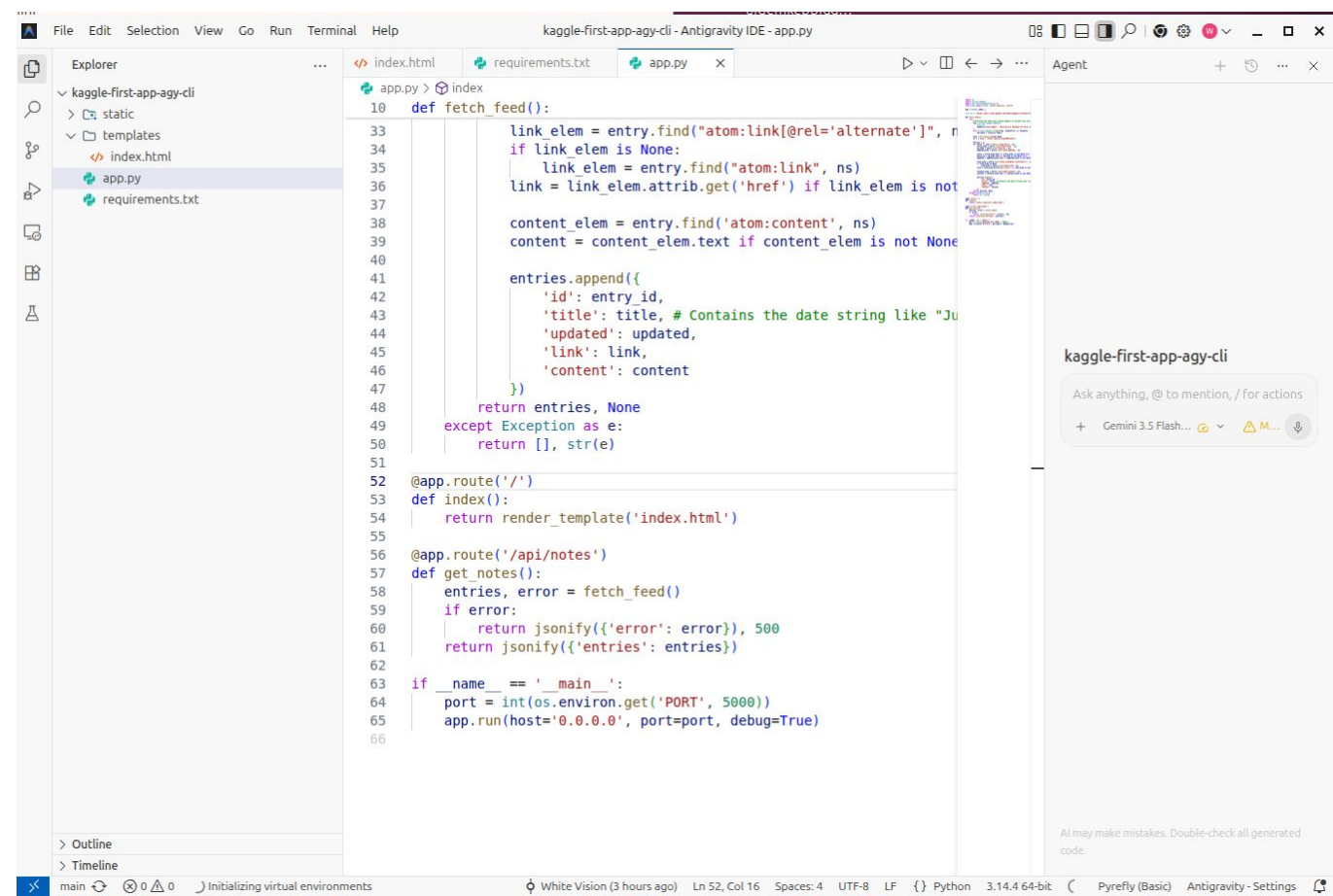


Ejemplos de Agentes: Gemini Gemini chat

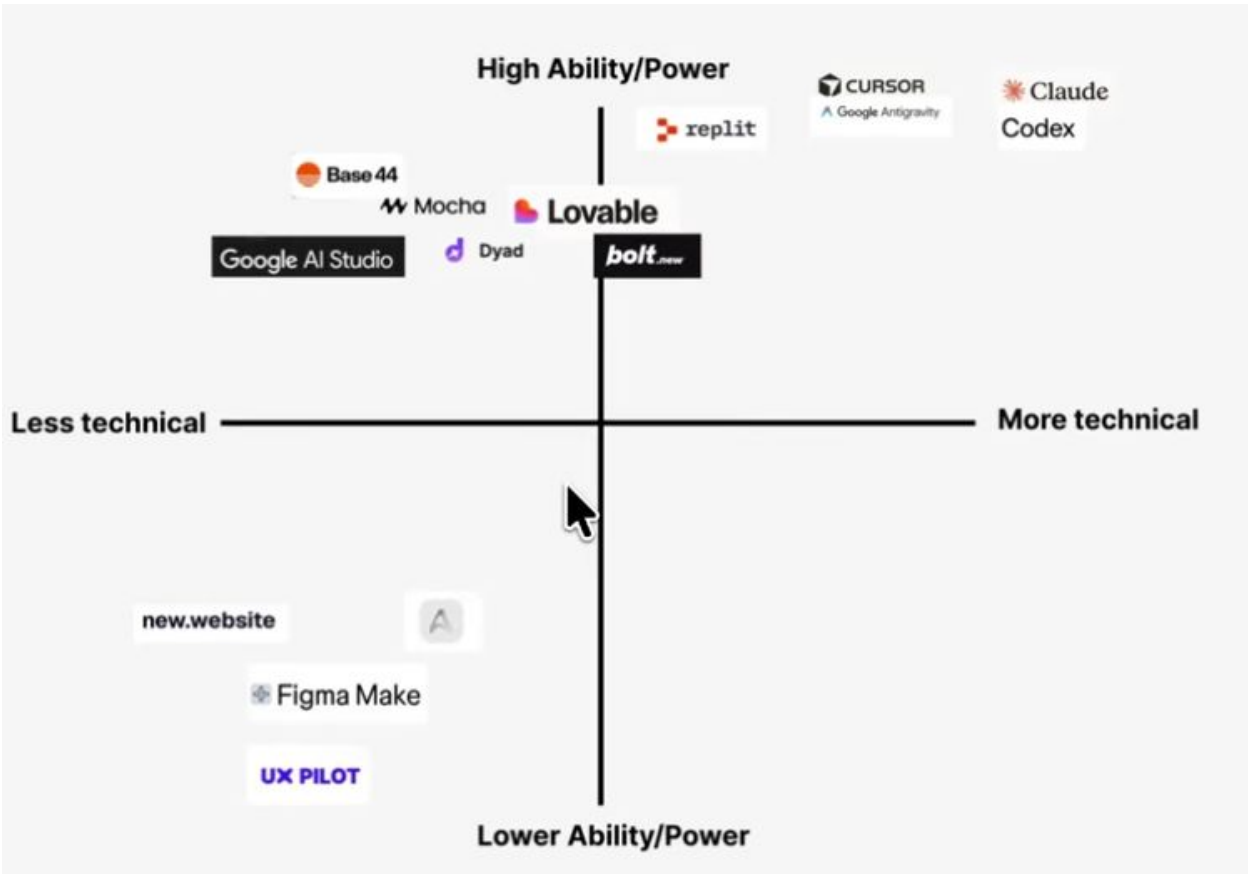


Gemini app: <https://gemini.google.com/app>

Google Antigravity IDE



Agentes para Programar (Codear)



Librerías para codear Agentes

<https://adk.dev/get-started/>

Agentes RAG: Framework: LlamaIndex

LlamaIndex proporciona las herramientas para construir casos de uso de **aumento de contexto**, desde el prototipo hasta la producción.

Cuenta con herramientas que permiten:

- ingerir,
- parsear,
- indexar,
- procesar datos,
- implementar rápidamente flujos de trabajo de consulta complejos combinando el acceso a los datos con el *prompting* de LLMs.
- armar agentes

El ejemplo más popular de aumento de contexto es la Generación Aumentada por Recuperación o RAG (por sus siglas en inglés), que combina el contexto con los LLMs en tiempo de inferencia.

<https://www.llamaindex.ai/>

<https://developers.llamaindex.ai/llamaparse/>

Agentes: Framework: LangChain

- Open-source framework para crear agentes
- Simplifica el desarrollo, despliegue y puesta en producción de aplicaciones que usen Modelos de Lenguaje (LLMs)
- Funcionalidades:
 - LangChain Expressions Language (LCEL) para componer y manejar cadenas (chains)
 - LangGraph para crear aplicaciones multiagentes
 - LangServe para crear APIs REST
 - LangSmith una plataforma de desarrollo para debuggear, testear y monitorear aplicaciones de LLMs
 - Integración con fuentes de datos para RAG

Agentes: Framework: Google ADK

- Open-source framework para crear agentes
- Creado por Google
- Ayuda en la creación, gestión, evaluación y despliegue de agentes impulsados por Inteligencia Artificial

Discover, build and deploy agents

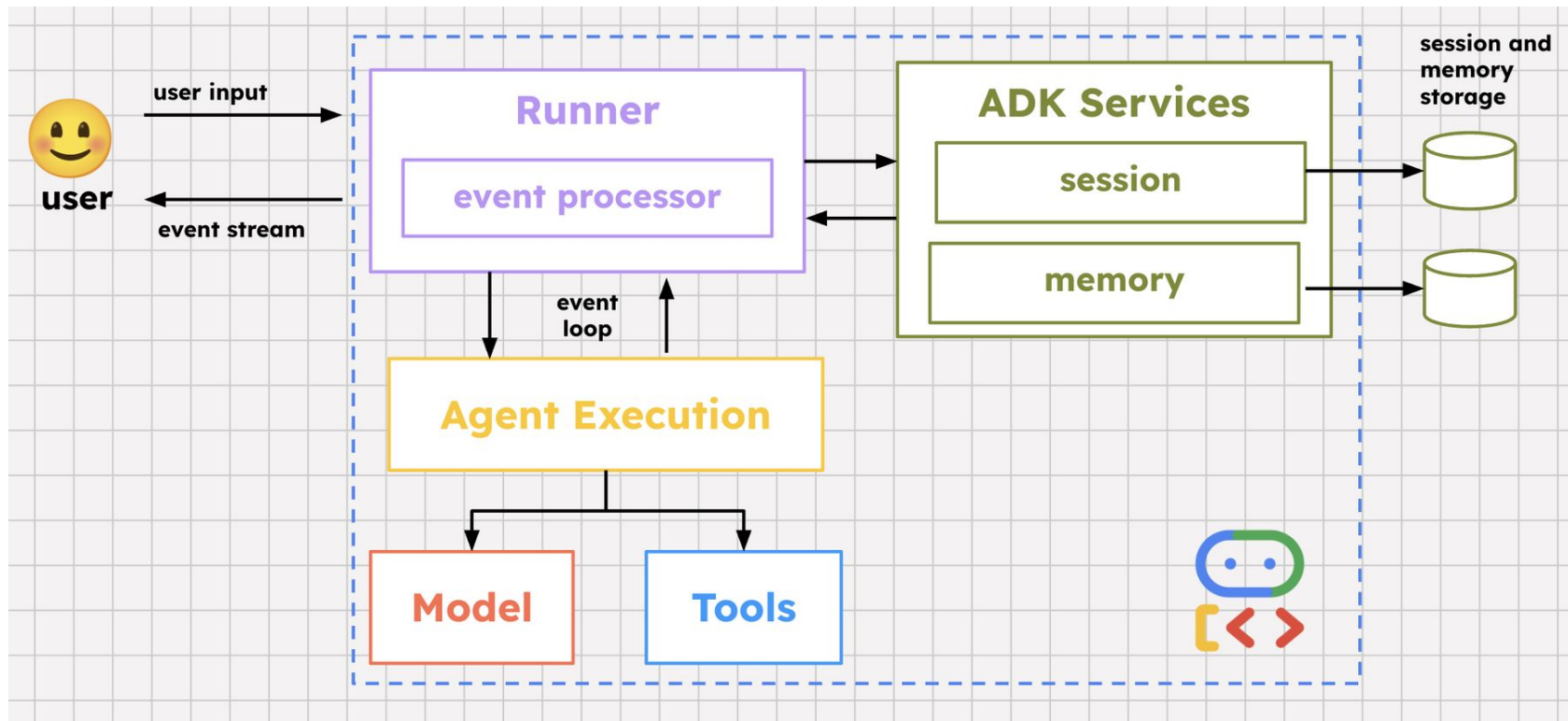


<https://adk.dev/get-started/about/>

Agentes: Frameworks - Comparativo

Herramienta / Framework	Enfoque Principal	Fortaleza Clave	Manejo de Datos	Flujo de Trabajo	Complejidad
ADK (Agentic Dev Kit)	Creación rápida de Agentes.	Abstracción de alto nivel para herramientas.	Utiliza los motores de LlamaIndex.	Basado en Tasks y Tools.	Baja/Media (Diseñado para rapidez).
CrewAI	Orquestación de sistemas multi-agente colaborativos.	Agentes con roles definidos que colaboran entre sí (Role-playing).	Delegado a herramientas (Se integra nativamente con LangChain/LlamaIndex).	Basado en Agents, Tasks y Crews (Procesos secuenciales o jerárquicos).	Media (Simplifica la lógica compleja de múltiples agentes interactuando).
LlamaIndex	Datos y Recuperación (RAG).	Indexación y búsqueda avanzada.	Excelente (Estructura datos complejos).	Basado en Query Engines.	Media (Fácil de empezar para RAG).
LangChain	Orquestación y Cadenas.	Ecosistema masivo e integraciones.	Genérico (Carga básica de documentos).	Basado en Chains y DAGs.	Alta (Mucha flexibilidad, curva lenta).
LangGraph	Flujos de trabajo cíclicos y agentes con estado (Stateful).	Ciclos (loops), memoria persistente y control detallado del flujo paso a paso.	Agnóstico (Basado en la actualización y paso de un "Estado" entre nodos).	Basado en Grafos (Nodos, Aristas y Estados condicionales).	Alta (Curva de aprendizaje empinada, ideal para control total sobre agentes).

Agentes: Frameworks - Google ADK (Agent Development Kit)



Google ADK <https://adk.dev/get-started/python/>
<https://cloud.google.com/blog/topics/developers-practitioners/remember-this-agent-state-and-memory-with-adk>

Componentes de Google ADK



Agente (Agent)

La unidad fundamental de trabajo diseñada para tareas específicas. Los agentes pueden usar modelos de lenguaje (LlmAgent) para el razonamiento complejo, o actuar como controladores deterministas de la ejecución, a los cuales se les llama "agentes de flujo de trabajo".



Herramienta (Tool)

Otorga a los agentes habilidades más allá de la conversación, permitiéndoles interactuar con APIs externas, buscar información, ejecutar código o invocar otros servicios externos al modelo base.



Memoria (Memory)

Permite a los agentes recordar información sobre un usuario a través de múltiples sesiones, proporcionando un contexto a largo plazo persistente (distinto del Estado de sesión a corto plazo).



Modelos

El LLM base que impulsa a los LlmAgents, procesando los tokens y habilitando las capacidades abstractas de comprensión, generación y razonamiento lógico.



Callbacks

Fragmentos de código personalizado que proporcionas para que se ejecuten en puntos específicos del proceso del agente, lo que permite realizar verificaciones, registros (logging) o modificaciones dinámicas de comportamiento.



Planificación (Planning)

Una capacidad cognitiva avanzada donde los agentes pueden desglosar objetivos de alto nivel en pasos lógicos más pequeños, evaluar rutas alternativas y organizar un cronograma de ejecución (ej. arquitecturas ReAct).

Componentes de Google ADK



Gestión de Sesiones

Maneja el contexto de una conversación individual (**Session**), incluyendo su historial (**Eventos**) y la memoria de trabajo temporal del agente para esa conversación (**Estado**).



Artefactos (Artifact)

Permite a los agentes guardar, cargar y gestionar archivos o datos binarios complejos (como imágenes, documentos PDFs, hojas de cálculo) asociados con una sesión o usuario particular.



Ejecución de Código

La capacidad de los agentes (generalmente a través de Herramientas especializadas) para generar, probar y ejecutar código en entornos seguros (sandbox) con el fin de realizar cálculos, análisis de datos o acciones complejas.



Evento (Event)

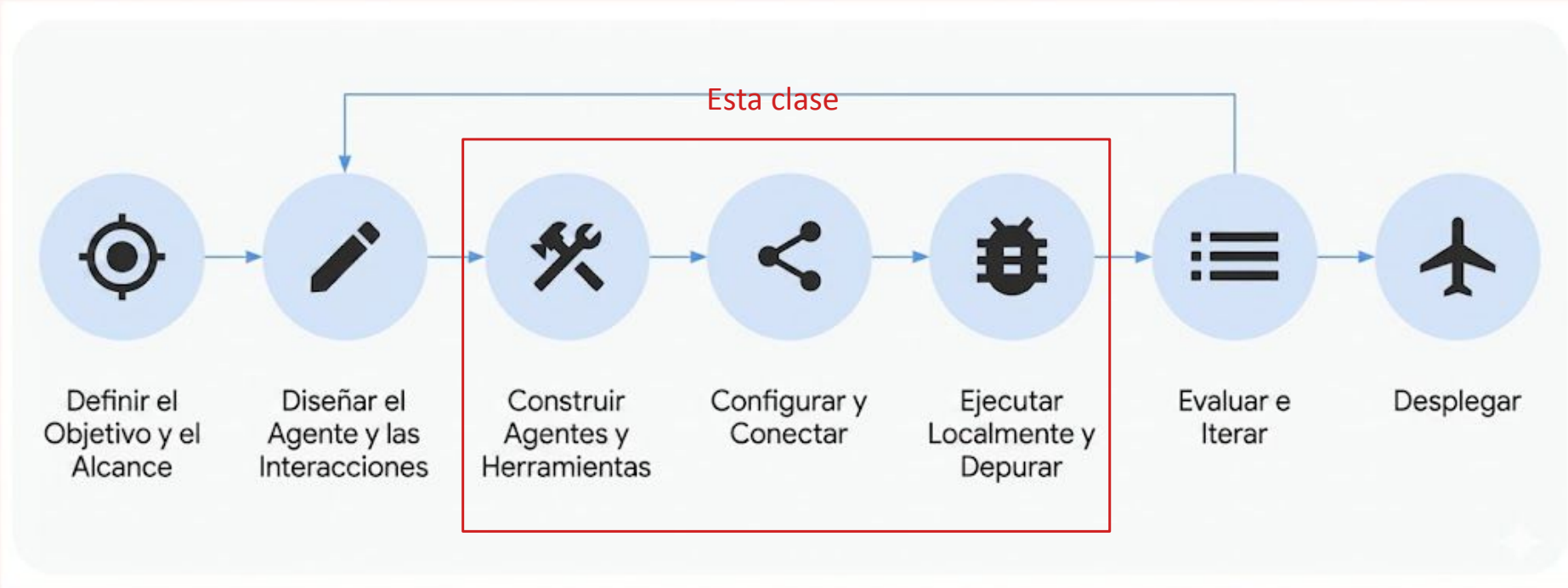
La unidad atómica de comunicación. Representa sucesos asíncronos en una sesión (inputs del usuario, outputs del agente, fallos de red), construyendo el grafo de la conversación e instruyendo la dirección del sistema.



Ejecutor (Runner)

El motor principal (Loop de control) que gestiona el flujo de ejecución, enruta los Eventos, orquesta la interacción entre múltiples agentes y se comunica con la infraestructura de backend de manera continua.

Proceso de Creación de Agentes

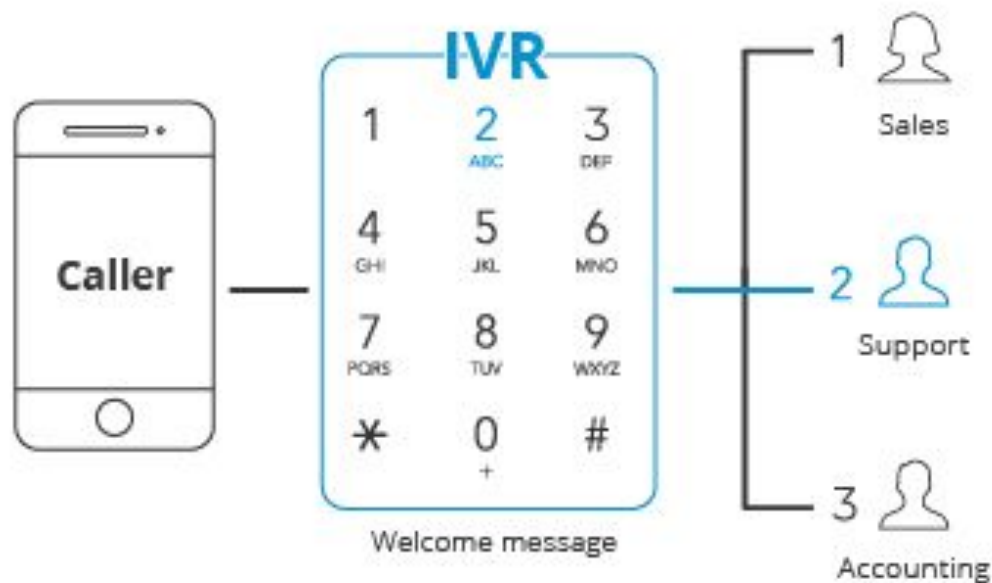


Ejemplos de Agentes: Chatbot Whatsapp



Fuente: <https://verge-ai.com/whatsapp-chatbot/>

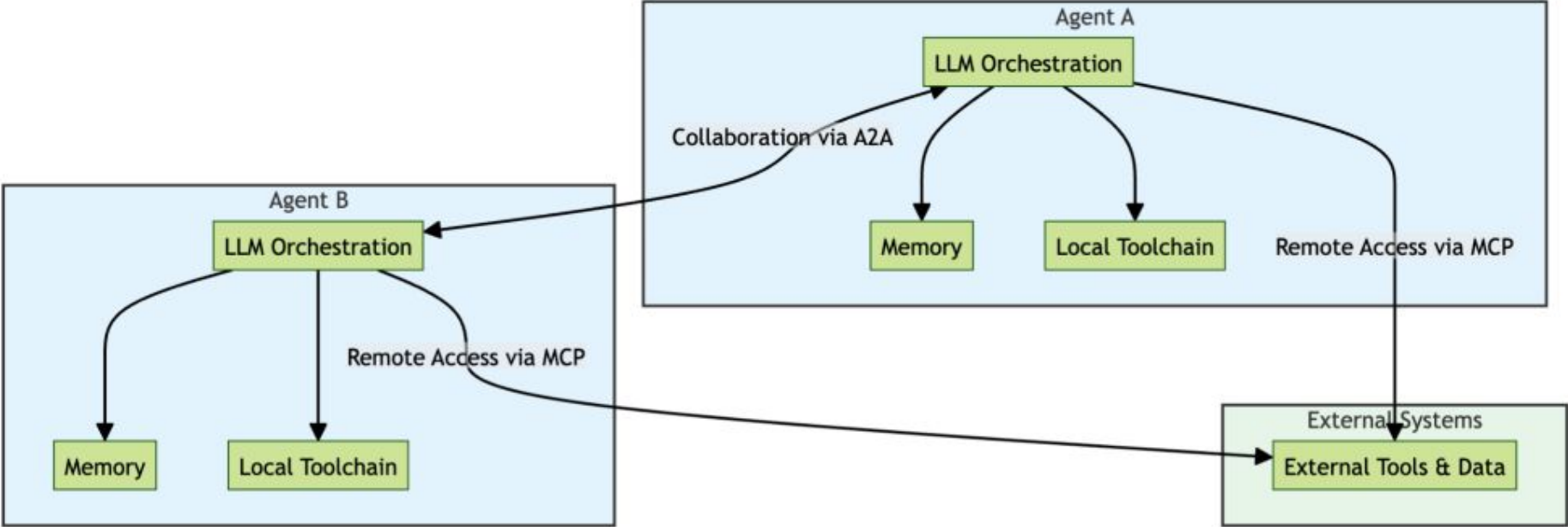
Ejemplos de Agentes: Evolución IVR (Interactive Voice Response)



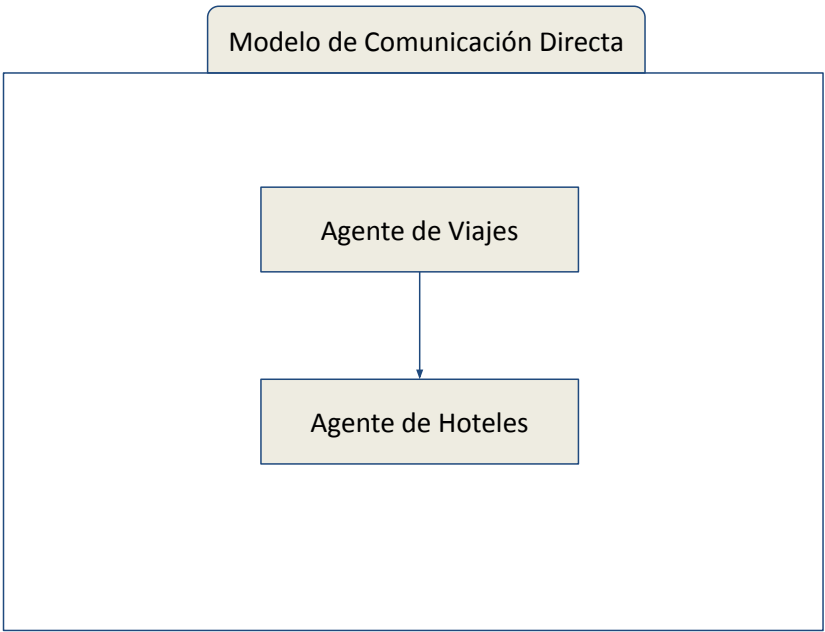
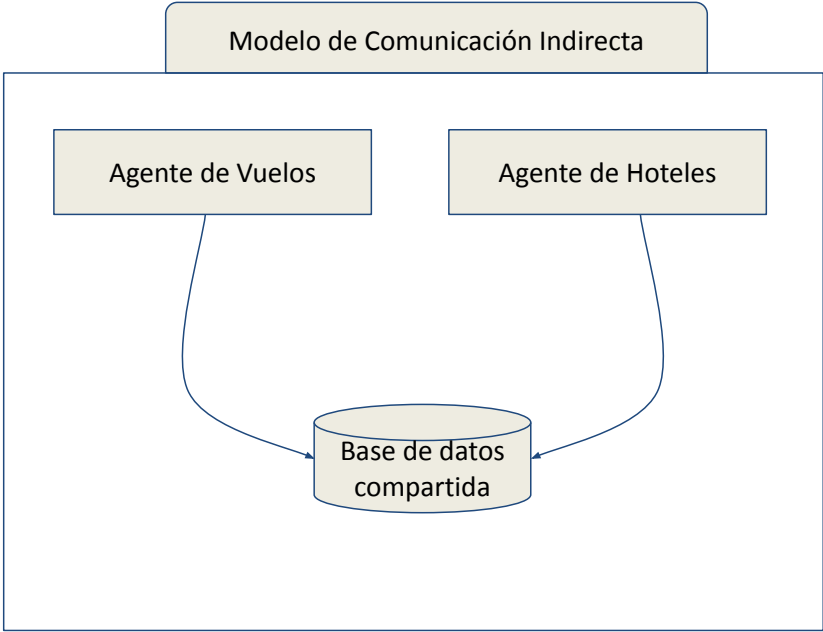
Los IVR utilizan NLP (Natural Language Processing) para determinar la intención del usuario. El surgimiento de los Agentes híbridos permite la creación de aplicaciones híbridas para la atención al cliente o atender llamadas. Surgen así nuevas plataformas de Agentes Conversacionales por voz.

Arquitecturas Multiagentes

Arquitectura Multi-Agente

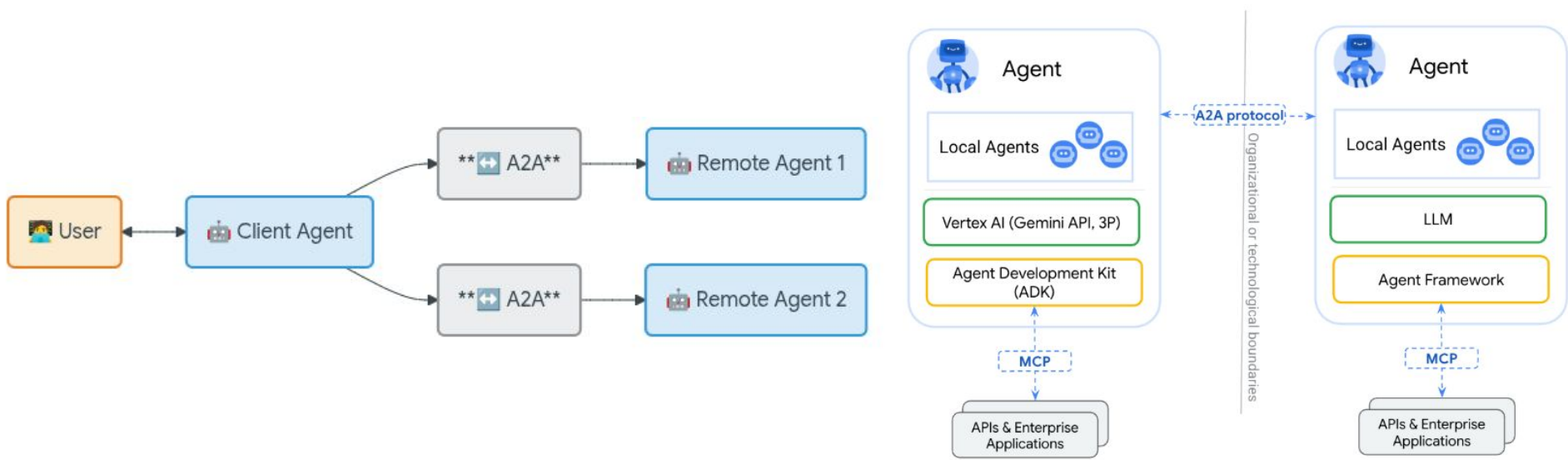


Interacción entre Agentes



A2A

A2A Agent to Agent - Se trata de un protocolo que permite la comunicación entre Agentes. Esto es particularmente útil si tenemos Agentes desarrollados con distinta tecnología y precisan comunicarse entre sí.

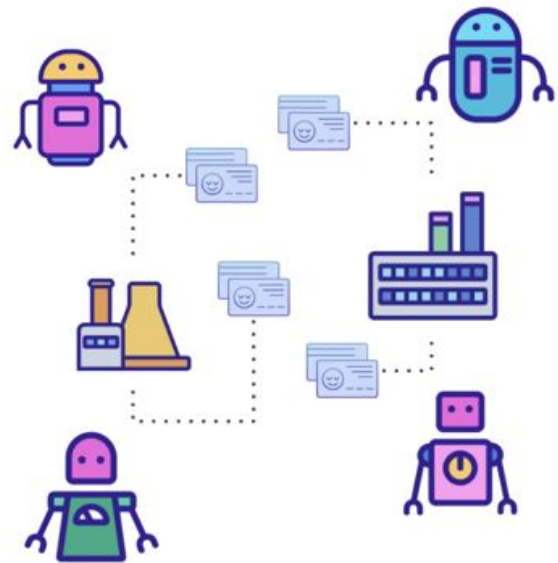


Agent-to-Agent <https://a2a-protocol.org/latest/>

A2A - Agent Card



A2A - Agent Card



Name: Agent ABC
Provider: Google (URL)
Preferred input/output: text

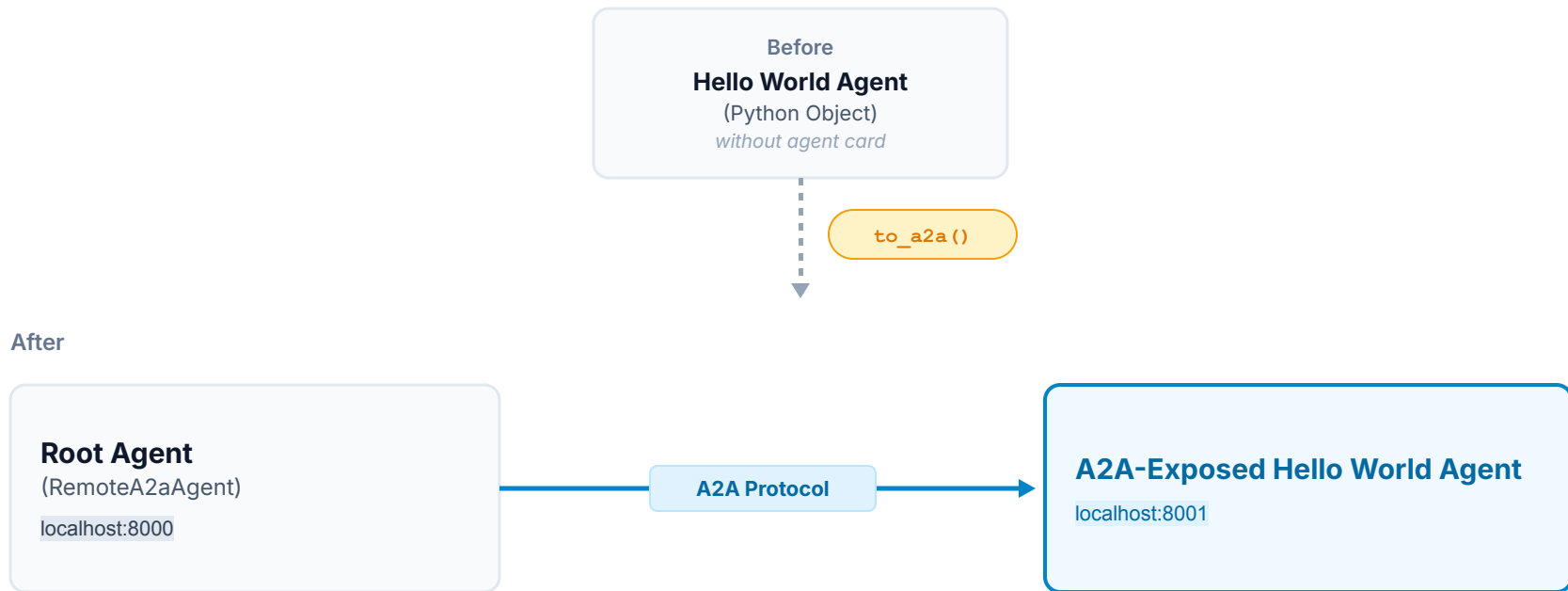
Capabilities:
I can do streaming
I can do Push Notifications

- Skills:**
- Skill A
 - Description
 - Input
 - Output
 - Skill B
 - Description
 - Input
 - Output

Authentication:
Scheme: Bearer
Key: xxx

<https://codelabs.developers.google.com/codelabs/currency-agent#0>

A2A - Google ADK



Documentation: <https://adk.dev/a2a/quickstart-exposing/>

Demo

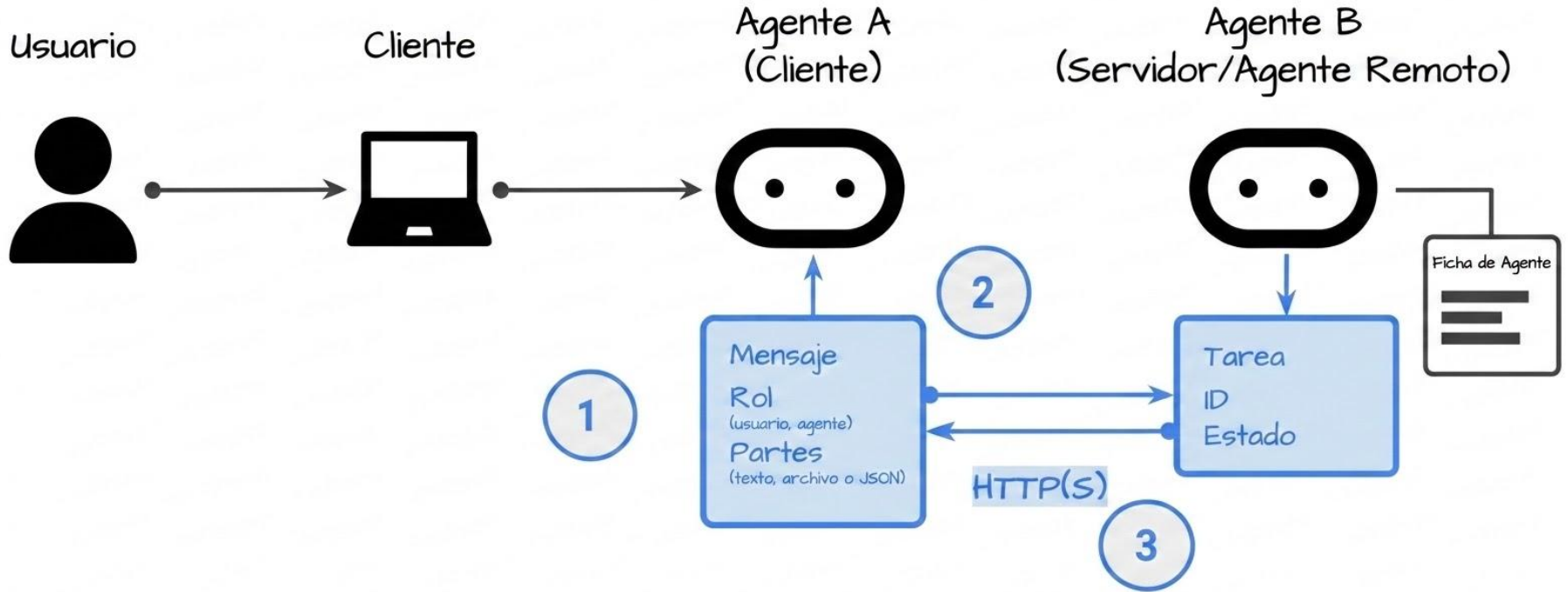
A2A - Agent-to-Agent

A2A - Librería: Conceptos Clave

Elemento	Descripción	Propósito Clave
Ficha de Agente Agent Card	Un documento de metadatos JSON que describe la identidad, capacidades, endpoint, habilidades y requisitos de autenticación de un agente.	Permite a los clientes descubrir agentes y comprender cómo interactuar con ellos de manera segura y efectiva.
Tarea Task	Una unidad de trabajo con estado iniciada por un agente, con un ID único y un ciclo de vida definido.	Facilita el seguimiento de operaciones de larga duración y permite colaboraciones e interacciones de múltiples turnos.
Mensaje Message	Un único turno de comunicación entre un cliente y un agente, que contiene contenido y un rol ("usuario" o "agente").	Transmite instrucciones, contexto, preguntas, respuestas o actualizaciones de estado que no son necesariamente formales.
Parte Part	El contenedor de contenido fundamental en Mensajes y Artefactos. Contiene texto, referencia de archivo (URL o bytes) o datos estructurados.	Proporciona flexibilidad a los agentes para intercambiar diversos tipos de contenido dentro de mensajes y artefactos.
Artefacto Artifact	Un resultado tangible generado por un agente durante una tarea (por ejemplo, un documento, imagen o datos estructurados).	Entrega los resultados concretos del trabajo de un agente, garantizando salidas estructuradas y recuperables.

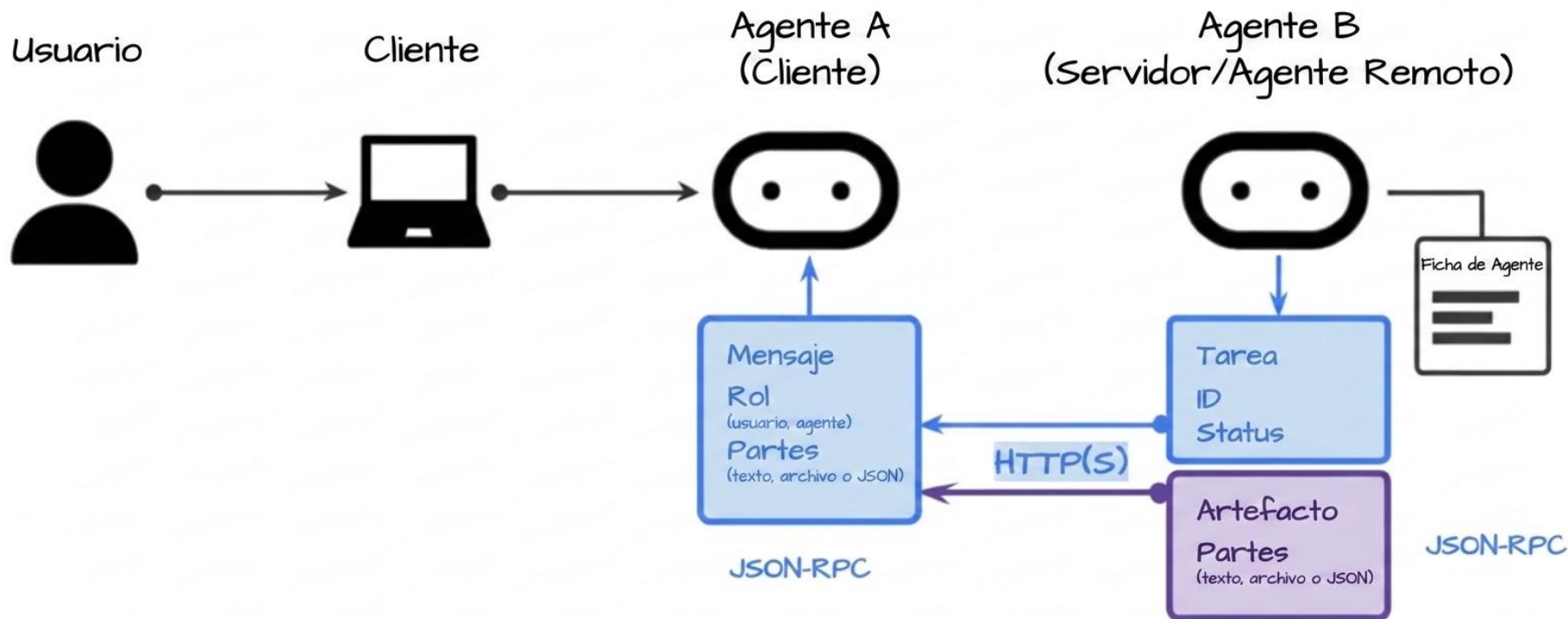
Documentación: <https://a2a-protocol.org/latest/topics/key-concepts/>

A2A - Comunicación - Polling

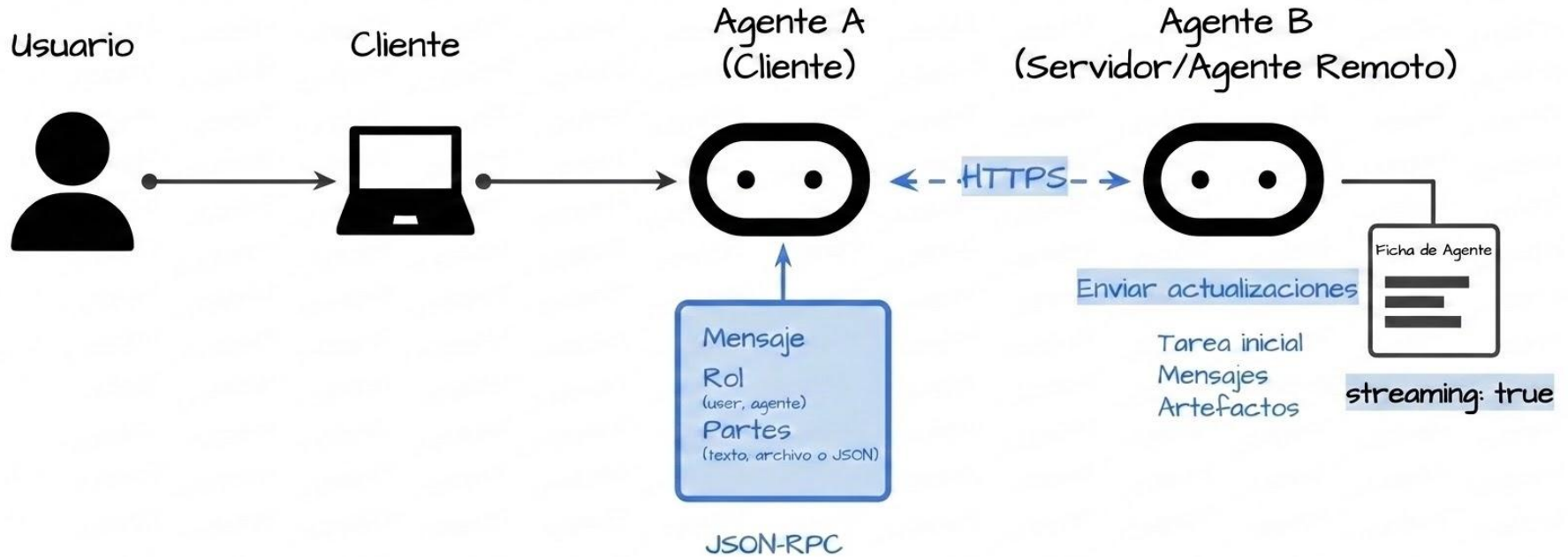


Como funciona A2A Video: https://www.youtube.com/watch?v=Fbr_Solax1w

A2A - Comunicación - Resultado



A2A - Comunicación - Streaming



<https://a2a-protocol.org/latest/tutorials/python/1-introduction/>
<https://docs.langchain.com/langsmith/server-a2a>

Demo

A2A - Agent-to-Agent

2

Niveles de Maduración

Podemos clasificar los Agentes desde distintas perspectivas, sin embargo, en este apartado vamos a clasificarlos según su complejidad.

A mayor autonomía y complejidad del sistema agéntico, mayores son los riesgos y controles que se precisan. También, va a estar relacionado con los casos de uso que busquemos resolver, un agente simple que se dedica a diagnosticar personas sin duda requerirá muchísimo control.

Definimos 6 niveles que iremos viendo a lo largo del curso:

1. Sistema básico agéntico (single agent)
2. Sistema dinámico de workflows de agentes (dynamic workflow agents)
3. Sistema agéntico con patrones introspectivos como React y Reflexion (react and reflexion agents)
4. Sistemas multi-agénticos (multi-agent)
5. Sistemas multiagénticos avanzados con meta-agentes (Advanced multi-agent)
6. Sistemas agénticos con mecanismos de feedback y auto-aprendizaje (Self-correcting agents)

Otras formas de clasificar patrones:

<https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system>

Nivel 4: Sistemas multi-agénticos (multi-agent)

multi-agent

semi-autónomo

Múltiples agentes especializados colaborando para llevar a cabo tareas complejas. Cada agente se focaliza en diferentes tareas no superpuestas. Su organización puede permitir ejecutar tareas en paralelo.

Escalabilidad: Ideal para entornos altamente escalables. Sus tareas se pueden distribuir en múltiples agentes, permitiendo el procesamiento paralelo de flujos complejos.

Complicaciones: Más complejo de manejar. Sistemas de monitoreo son requeridos para asegurarse de que todo el sistema semiautónomo de agentes colabore en una forma que se adhiera a las regulaciones

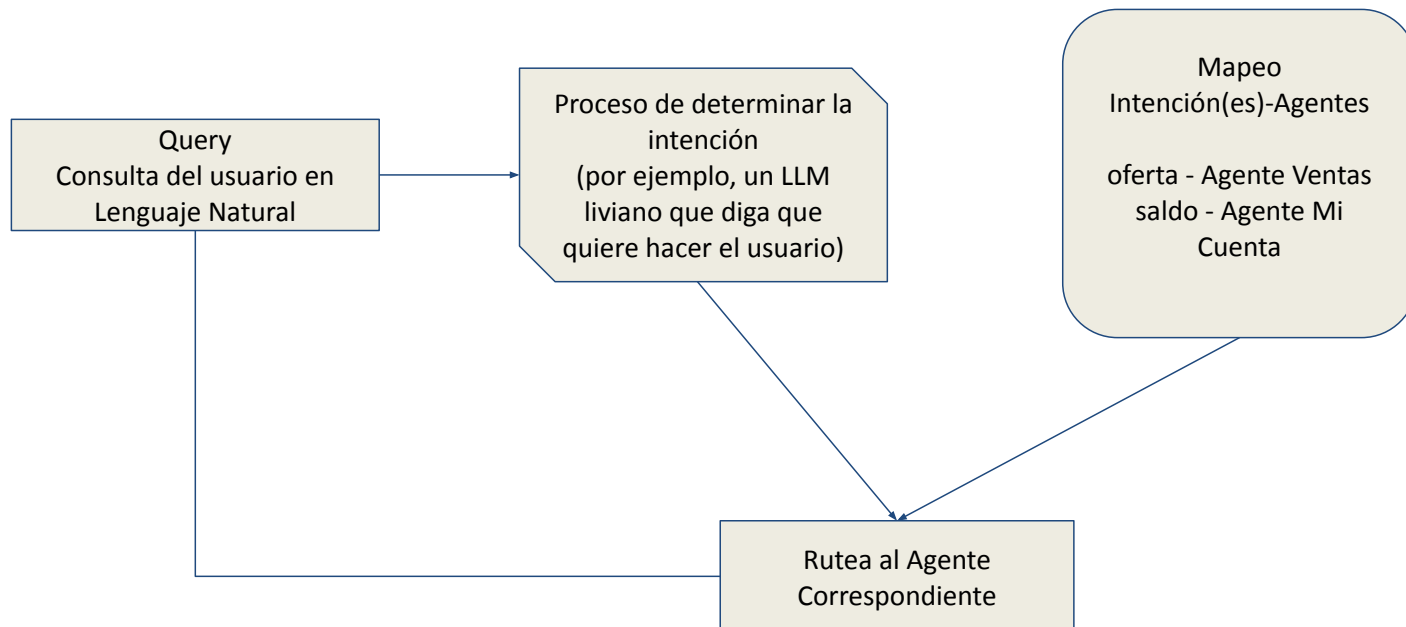
Implementación (Patrones):

- [Arquitectura Supervisor](#)
- [Planificación Multi-Agente \(multi-agent planning\)](#)
- [Shared Epistemic Memory](#)
- [Event Driven Reactivity](#)
- [Tool/Agent Registry](#)

Agent Router

Agent Router pattern
(ruteo basado de la intención)

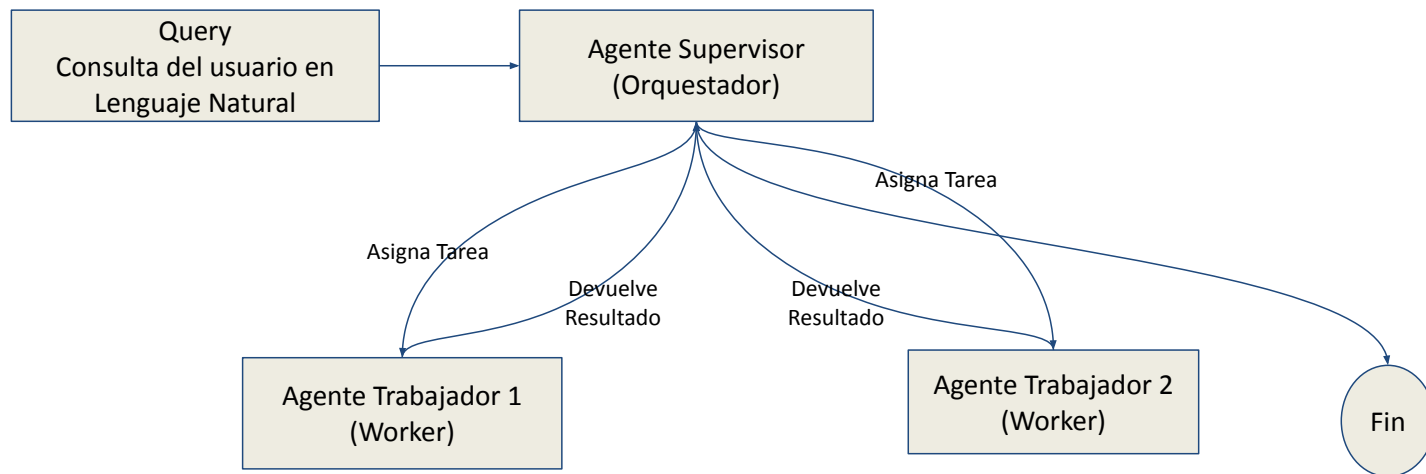
El agente detecta la intención (lo que quiere hacer el usuario) y redirecciona al agente que va a ejecutar lo necesario.



Supervisor Architecture

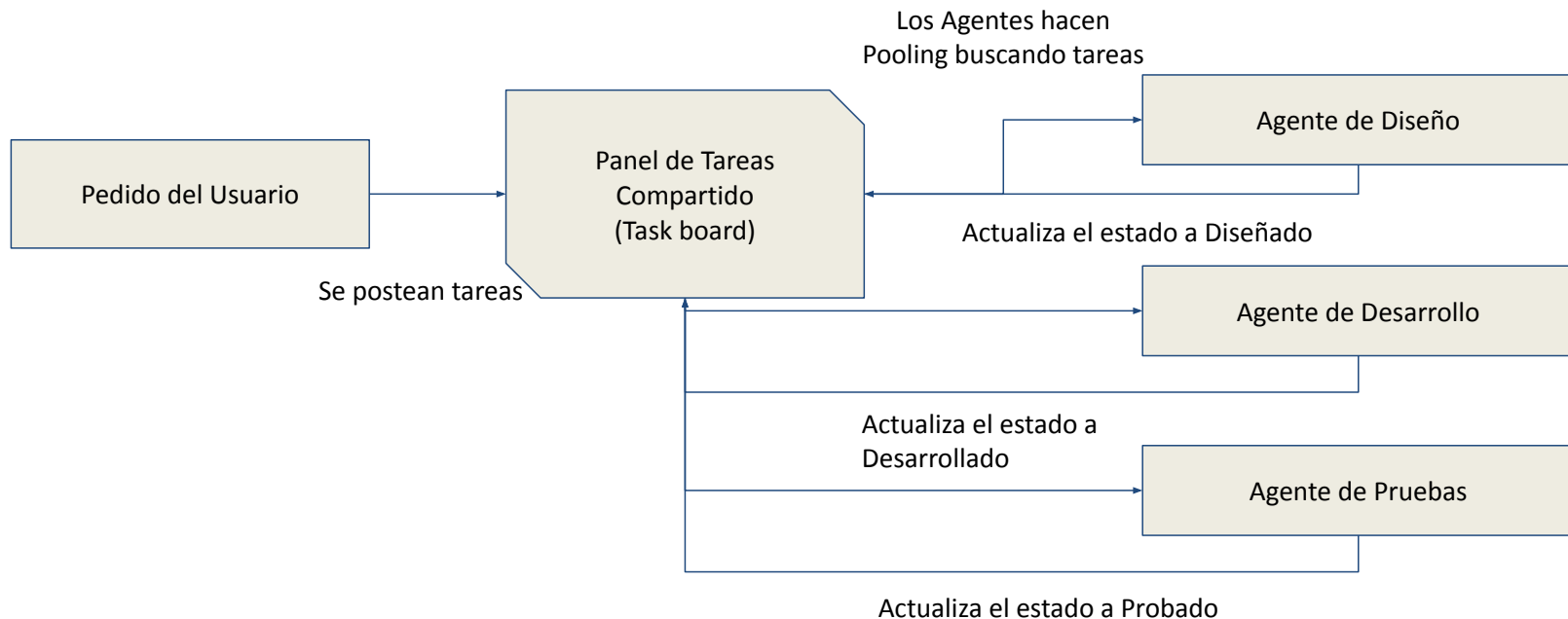
Un agente central que delega en el resto de los agentes.

Contexto: Una tarea compleja requiere la ejecución de muchos pasos secuenciales y el sistema necesita garantizar que el flujo se ejecute correctamente.



Swarm Architecture

Se basa en comunicación compartida o un panel de tareas. Una tarea se poste, y cualquier agente la toma y trabaja en ella. Una vez que el agente termina, actualizar el estado de la tarea, disponibilizándose al próximo agente en el workflow.



Nivel 5: Sistemas multiagénticos avanzados con meta-agentes (Advanced multi-agent)

multi-agent

meta-agents

semi-autónomo

Un “meta-agente” es introducido para supervisar y coordinar a los otros agentes. Esto permite que haya una reasignación dinámica de tareas y ajuste del plan en tiempo real.

Escalabilidad: adaptabilidad mejorada. El sistema puede escalar efectivamente incluso en entornos cambiantes debido a la optimización en la distribución de las tareas por el meta-agente

Complicaciones: Meta-agentes actúan como supervisores, ayudando a mantener la adherencia a la política ajustando los flujos de trabajo (workflows) y robusteciendo las tareas necesarias.

Implementación (Patrones):

- Meta-agents
- Blackboard Topology
- Resource Allocation
- Contract-Net Marketplace
- Supervision Trees

Nivel 6: Sistemas agénticos con mecanismos de feedback y auto-aprendizaje (Self-correcting agents)

multi-agent

feedback loops

self-correction

Avanzados sistemas multiagentes usan feedback loops complejos y de turnos múltiples. Los agentes critican, corrigen y redefinen las salidas de los otros iterativamente, llevando a un proceso de mejora continua

Escalabilidad: altamente escalables con mejora continua incorporada. Estos sistemas pueden evolucionar en tiempo real, haciéndose extremadamente efectivos para tareas dinámicas.

Complicaciones: Más complejo. Verificación de chequeos de cumplimiento (compliance checks) automatizados y autocorrección de acciones necesarias para asegurarse de que los agentes sigan alineados a las políticas mientras se adaptan.

Implementación (Patrones/Técnicas):

- Consensus
- Agent Negotiation
- Conflict Resolution
- Fractal CoT
- Coevolved Agent Training
- Trust Decay



Construir Agentes y Herramientas

Modelos



Direct String / Registro

Diseñado para modelos estrechamente integrados con Google Cloud.

- Gemini y Claude
- Agent Platform
- Gemma con Google AI Studio
- Resolución directa a través del registro interno de ADK



Conectores de Modelos

Para una mayor compatibilidad con modelos fuera de Google.

- Apigee y LiteLLM
- Ollama, vLLM y LiteRT-LM
- Utiliza clases wrapper personalizadas



Ruta de Modelos

Selección dinámica de modelos en tiempo de ejecución.

- Función router inteligente
- Conmutación por error (failover) automática
- Optimiza la disponibilidad y costos

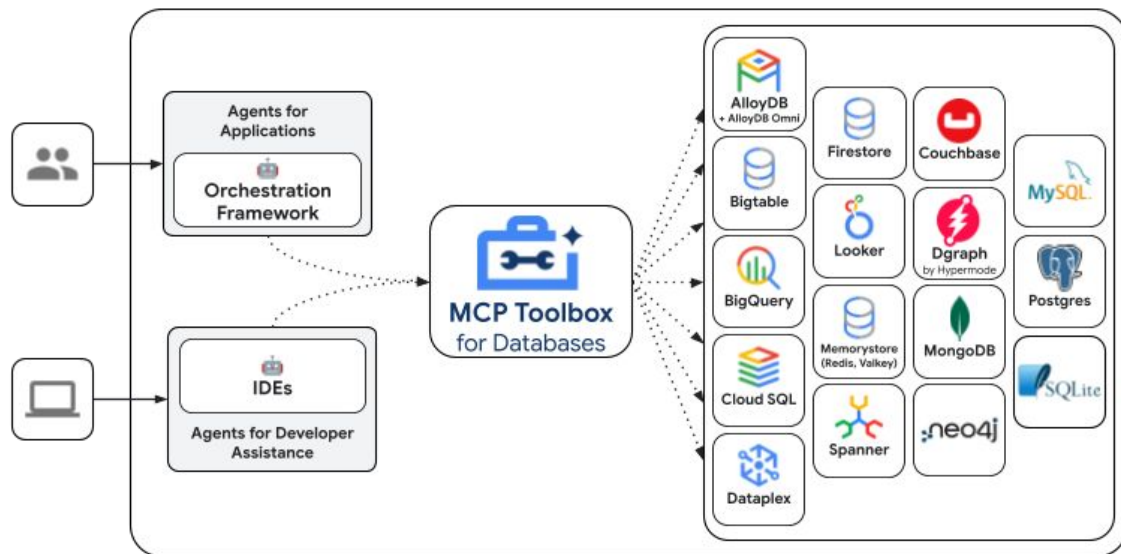
Langchain: <https://docs.langchain.com/oss/python/langchain/models>

Herramientas

Podemos usar herramientas para:

- Consultar bases de datos
- Hacer solicitudes API: obtener datos del clima, sistemas de reservas
- Buscar en la web
- Ejecutar fragmentos de código
- Recuperar información de documentos (RAG)
- Interactuar con otro software o servicios

y mucho más.



Herramientas

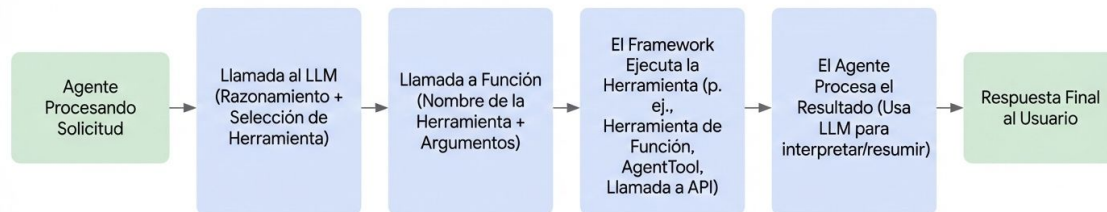
ADK trae una variedad de opciones para crear herramientas y para conectarse a otras ya creadas

La creación puede ser:

- [Function Tools](#)
- [Long Running Function Tools](#)
- [Agents-as-a-Tool](#)

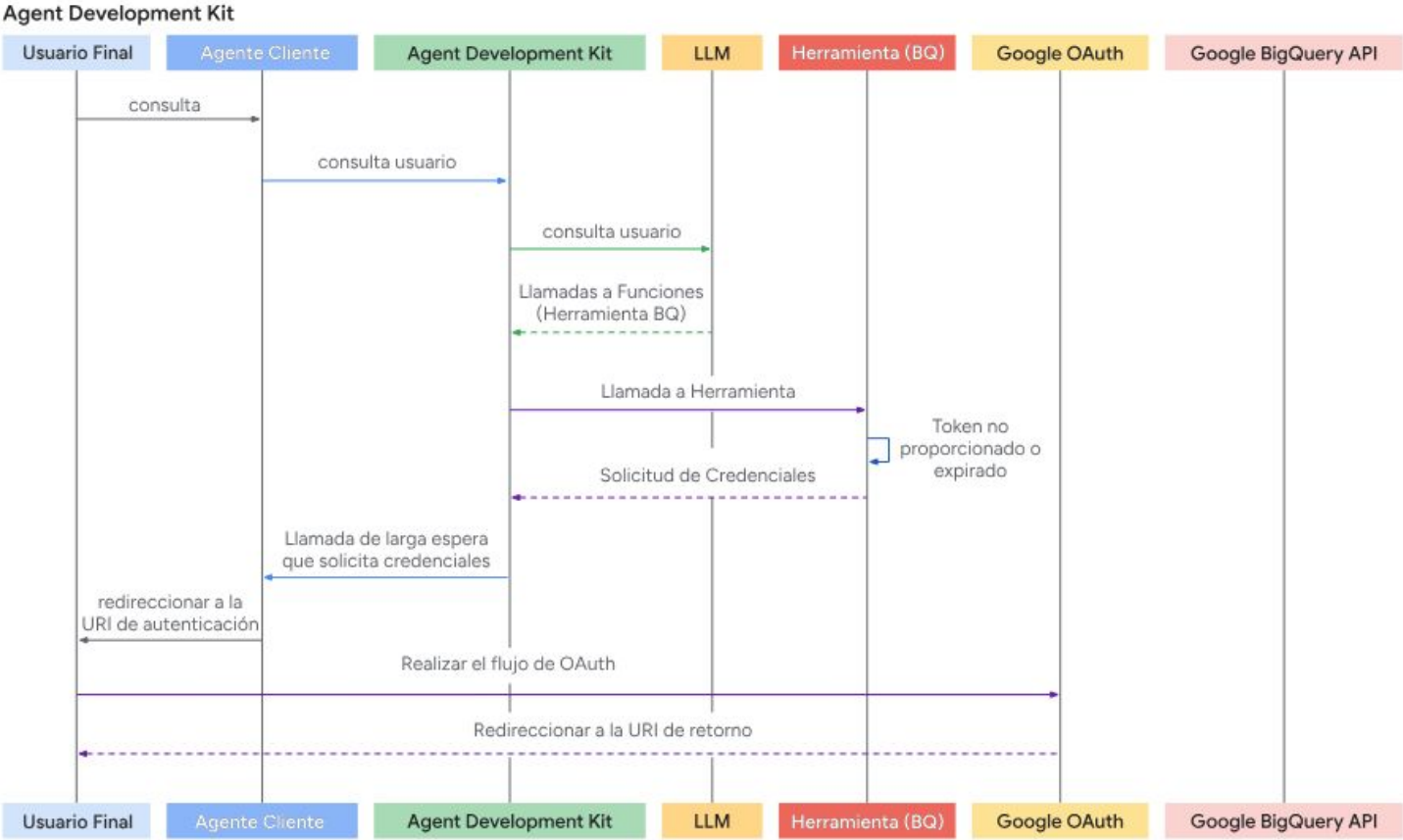
También puede usar:

- Herramientas de MCP
- Herramientas en formato OpenAPI

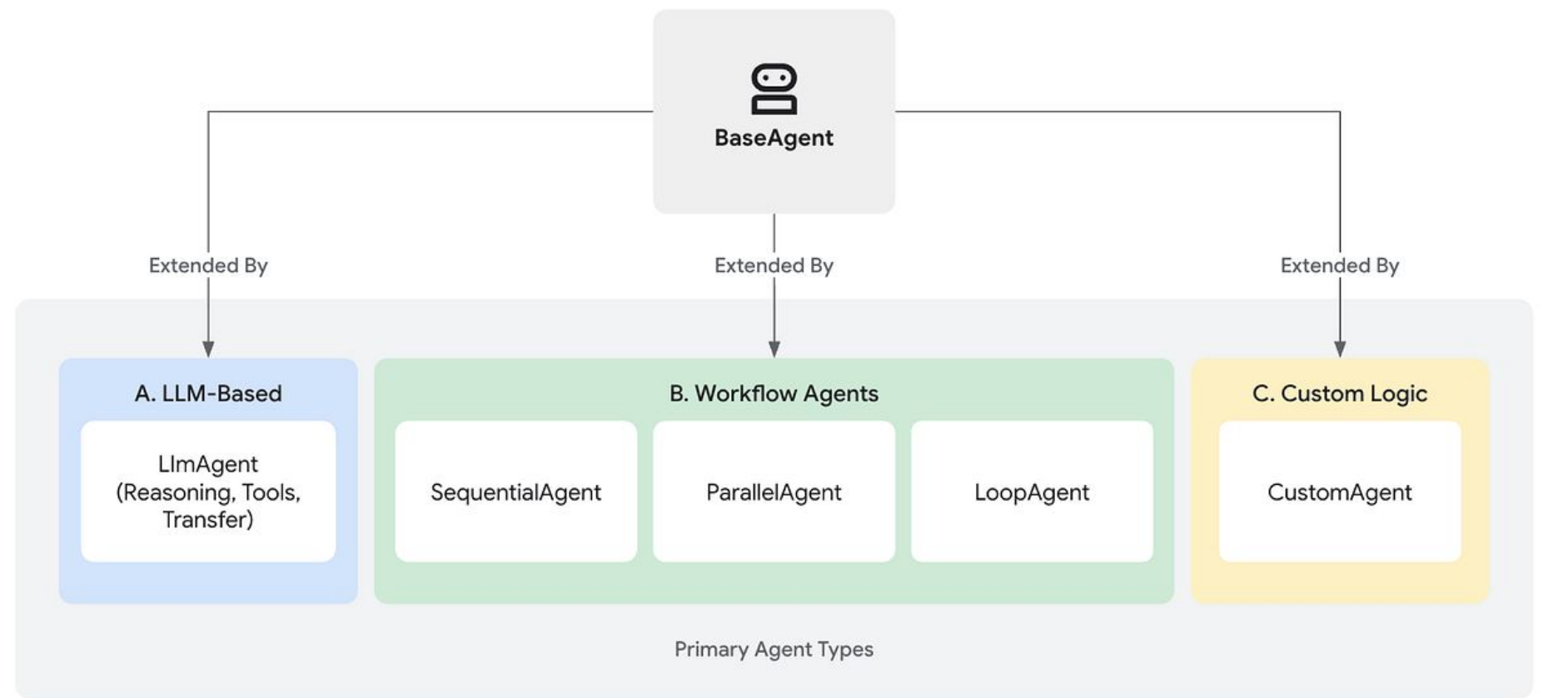


La autenticación va a ser importante, es decir, que el usuario tenga los permisos para ejecutar la herramienta. Para eso, se usa el protocolo OAuth

Proceso de Autenticación



Tipos de Agentes



LLM-Based

```
# Define una función de herramienta
def get_capital_city(country: str) -> str:
    """Recupera la ciudad capital para un país dado."""
    # Reemplaza con lógica real (ej., llamada API, búsqueda en base de datos)
    capitals = {"france": "Paris", "japan": "Tokyo", "canada": "Ottawa"}
    return capitals.get(country.lower(), f"Lo siento, no conozco la capital de {country}.")

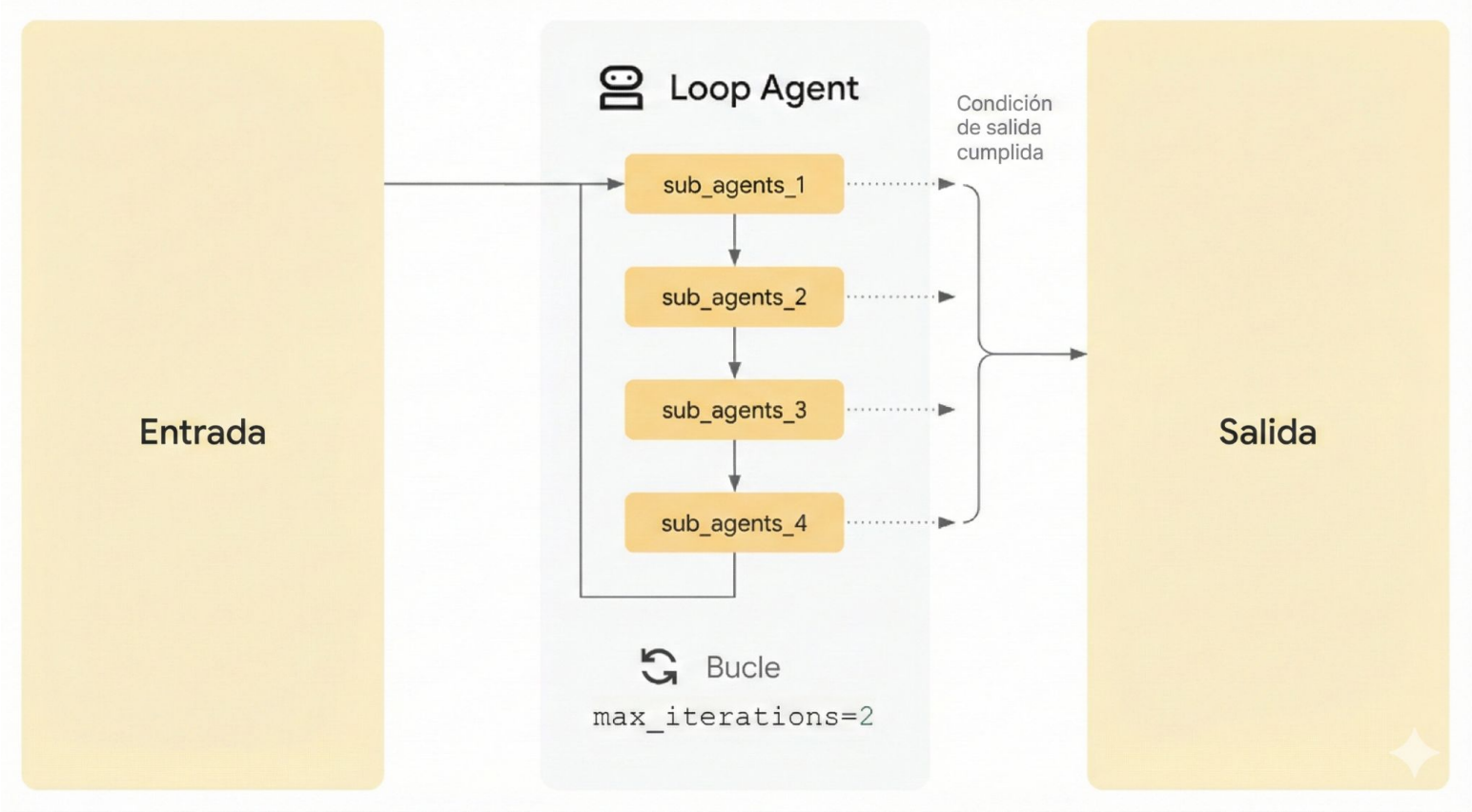
# Agrega la herramienta al agente
capital_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="capital_agent",
    description="Responde preguntas de usuarios sobre la ciudad capital de un país dado.",
    instruction="""Eres un agente que proporciona la ciudad capital de un país... (texto de instrucción anterior)""",
    tools=[get_capital_city] # Proporciona la función directamente
)
```

Sequential Agents

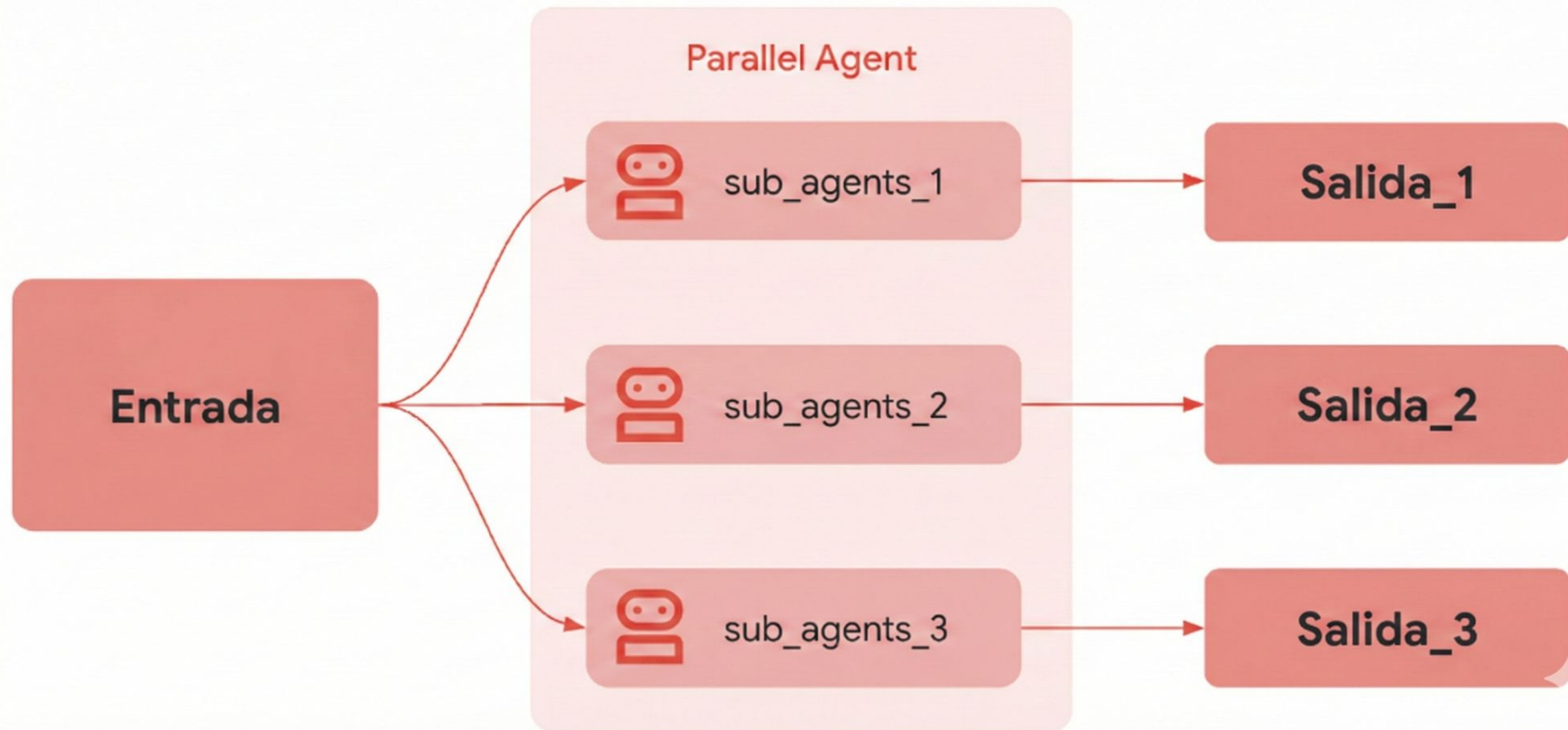


<https://adk-es.fabian-castro-c.dev/agents/workflow-agents/sequential-agents/#ejemplo>

Loop Agents



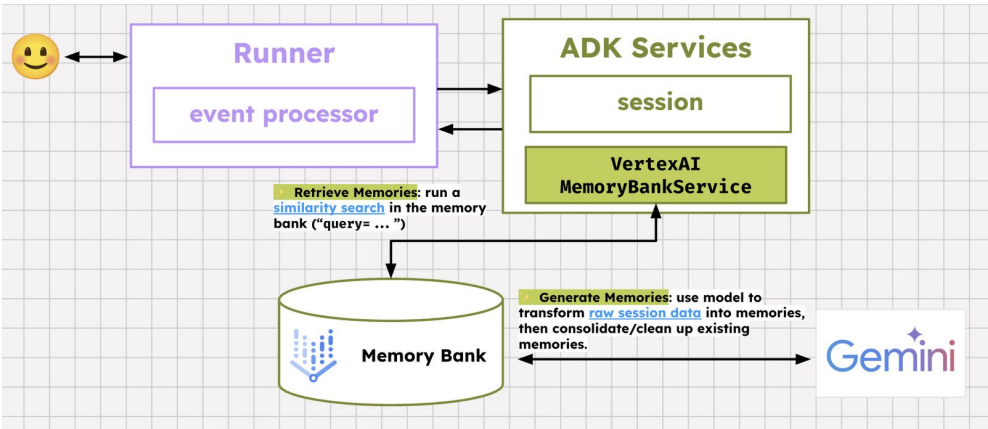
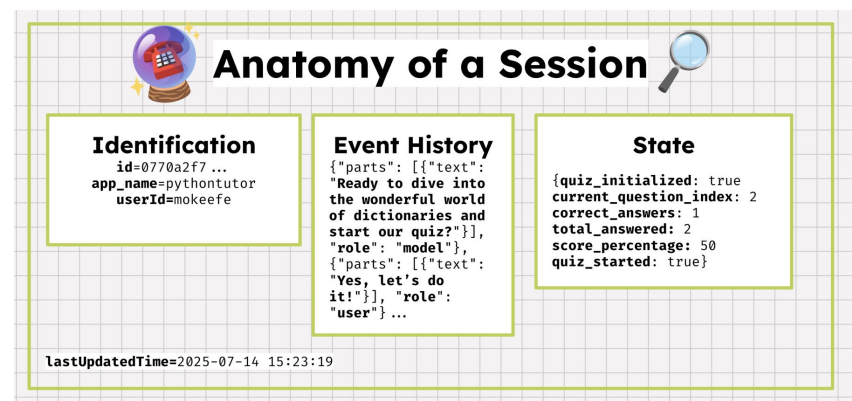
Parallel Agents



<https://adk-es.fabian-castro-c.dev/agents/workflow-agents/parallel-agents/#como-funciona>

Memoria

- **Session & State:** Se centran en la **interacción actual** – el historial y datos de la *única conversación activa*. Gestionados principalmente por un `SessionService`.
- **Memory:** Se centra en la **información pasada y externa** – un *archivo consultable* que potencialmente abarca múltiples conversaciones. Gestionado por un `MemoryService`.



<https://cloud.google.com/blog/topics/developers-practitioners/remember-this-agent-state-and-memory-with-adk>

Demo

Ejemplos de Agentes con Google ADK

<https://github.com/google/adk-samples>



Ejecutar Localmente y Depurar

ADK web

 **Agent Development Kit**

my_agent

+



Trace

Events

State

Artifacts

Sessions

Eval

Invocations

what can you do?

▼

what time is it in Paris?

▼

SESSION ID

a2580a79-ce58-4bfa-ab6f-f4725bf02a02

USER ID

user user

Token Streaming

+

New Session







I can tell you the current time in a specified city.



what can you do?



 get_current_time



 get_current_time



The current time in Paris is 10:30 AM.

what time is it in San Francisco?









Agent Development Kit

blogger_agent

+

Trace

State

Artifacts

Sessions

Eval

Invocations

plan a blog post about visiting tokyo

Invocation ID:
e-4c1f41f1-fd3c-4ff6-b539-c7e5ebf67930

|→ invocation

29599.06ms

invoke_agent interact...

29580.90ms

call_llm

29577.38ms

|→ generate_content...

29576.86ms

execute_tool ...

0.58ms

invoke_agent ...

27274.40ms

invokeAgen...

27252.08ms

call_llm

27236.55ms

|→ genera...

27236.31ms

invokeAgen...

10

SESSION ID 56bcabc1-9aee-42eb USER ID user Token Streaming | + New Session

#1

plan a blog post about visiting tokyo

#2

⚡ transfer_to_agent

#3

✓ transfer_to_agent

Here is a blog post outline for visiting Tokyo:

The Ultimate Tokyo Adventure: Your G

I. Introduction

• **Welcome to Tokyo:** A city where ancient traditions beautifully intertwine v temples, and culinary delights.

• **What You'll Discover:** This guide will take you through Tokyo's iconic neig navigate this incredible metropolis.

II. Exploring Tokyo's Iconic Neighborhoods & Distric

Dive into the unique atmosphere of Tokyo's most famous areas, each offering a c

Type a Message...

ADK Web Developer UI

ADK web no es una herramienta productiva, vamos a tener que crearle una interfaz al agente.

Con lo cual, si queremos que otros usen el agente tendríamos que armar un frontend o integrarlo a una página web u otra aplicación.

Existen algunas librerías para crear interfaces de Agente iniciales: streamlit, gradio. También, se pueden hacer custom.

Vibe Coding & Agentic Engineering



Agent Skills

Es un simple formato abierto que permite extender y especializar con conocimientos los agentes. Se trata de una carpeta con un archivo markdown [SKILLS.md](#). También, se incluyen subdirectorios con información especializada.

```
my-skill/
├─ SKILL.md           # Required: metadata + instructions
├─ scripts/           # Optional: executable code
├─ references/         # Optional: documentation
├─ assets/             # Optional: templates, resources
└─ ...                 # Any additional files or directories
```

Esta organización permite también reutilizar aquellas cosas que funcionaron.

Open Standard: <https://agentskills.io/home>

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

Agent Skills - Formato de los archivos - SKILL.md

Consta de dos partes:

- **Frontmatter** en YAML
- **Cuerpo** en Markdown

Frontmatter en YAML

Esta es la capa de metadatos. Es la única parte de la habilidad que es indexada por el enrutador de alto nivel del agente. Cuando un usuario envía un *prompt*, el agente realiza una coincidencia semántica del *prompt* con los campos de descripción de todas las habilidades disponibles.

Campos clave:

- **name:** No es obligatorio. Debe ser único dentro del alcance (*scope*). Solo en minúsculas y se permiten guiones (por ejemplo, *postgres-query*, *pr-reviewer*). Si no se proporciona, tomará por defecto el nombre del directorio.
- **description:** Es obligatorio y es el campo más importante. Funciona como la "frase de activación". Debe ser lo suficientemente descriptivo para que el LLM (Modelo de Lenguaje Grande) reconozca su relevancia semántica. Una descripción vaga como "Herramientas de base de datos" es insuficiente. Una descripción precisa como *"Ejecuta consultas SQL de solo lectura contra la base de datos PostgreSQL local para recuperar datos de usuarios o transacciones. Usa esto para depurar estados de datos"* asegura que la habilidad sea seleccionada correctamente.

A simple SKILL.md file

pdf/SKILL.md

YAML Frontmatter

```
---
name: pdf
description: Comprehensive PDF toolkit for extracting text and tables,
merging/splitting documents, and filling-out forms.
---
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see *./reference.md*. If you need to fill out a PDF form, read *./forms.md* and follow its instructions.

Quick Start

```
...
python
from pypdf import PdfReader, PdfWriter
```

```
# Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

```
...
```

A SKILL.md file must begin with YAML Frontmatter that contains a file name and description, which is loaded into its system prompt at startup.

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

Agent Skills - Formato de los archivos - SKILL.md

El cuerpo en Markdown

El cuerpo contiene las instrucciones. Esto es "ingeniería de *prompts*" persistida en un archivo. Cuando se activa la habilidad, este contenido se inyecta en la ventana de contexto del agente.

El cuerpo debe incluir:

- **Goal (Objetivo):** Una declaración clara de lo que logra la habilidad.
- **Instructions (Instrucciones):** Lógica paso a paso.
- **Examples (Ejemplos):** Ejemplos de entradas y salidas (*few-shot*) para guiar el rendimiento del modelo.
- **Constraints (Restricciones):** Reglas sobre lo que "no se debe hacer" (por ejemplo, "No ejecutar consultas DELETE").

Si usan Antigravity or Antigravity CLI, se copia en
<project-root>/.agents/skills/ (project scope).

Global skills: ~/ .gemini/antigravity-cli/skills/.

A simple SKILL.md file

pdf/SKILL.md

YAML Frontmatter

```
---
name: pdf
description: Comprehensive PDF toolkit for extracting text and tables,
merging/splitting documents, and filling-out forms.
---
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see ./reference.md. If you need to fill out a PDF form, read ./forms.md and follow its instructions.

Quick Start

```
'''
python
from pypdf import PdfReader, PdfWriter
```

```
# Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

```
'''
```

A SKILL.md file must begin with YAML Frontmatter that contains a file name and description, which is loaded into its system prompt at startup.

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

Agent Skills - Formato de los archivos - [SKILL.md](#) Referencias

Bundling additional content

pdf/SKILL.md

YAML Frontmatter

```
name: pdf
description: Comprehensive PDF toolkit for extracting text and
tables, merging/splitting documents, and filling-out forms.
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `./reference.md`. If you need to fill out a PDF form, read `./forms.md` and follow its instructions.

Quick Start

```
...
python
from pypdf import PdfReader, PdfWriter
```

```
# Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

```
# Extract text
text = ""
for page in reader.pages:
    text += page.extract_text()
...
```

pdf/reference.md

PDF Processing Advanced Reference

This document contains advanced PDF processing features, detailed examples, and additional libraries not covered in the main skill instructions.

pypdfium2 Library (Apache/BSO License)

Overview

pypdfium2 is a Python binding for PDFium (Chromium's PDF library). It's excellent for fast PDF rendering, image generation, and serves as ...

pdf/forms.md

If you need to fill out a PDF form, first check to see if the PDF has fillable form fields. Run this script from this file's directory:

```
'python scripts/check_fillable_fields <file.pdf>',
```

and depending on the result go to either the 'Fillable fields' or 'Non-fillable fields' and follow those instructions.

Fillable fields

If the PDF has fillable form fields:

- Run this script from this file's directory:

```
'python scripts/extract_form_field_info.py <input.pdf> <fields.json>'.
...
```

You can incorporate more context (via additional files) into your skill that can then be triggered by Claude based on the system prompt.

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

Agent Skills - También se pueden incluir scripts

Bundling executable scripts

pdf/forms.md

```
If you need to fill out a PDF form, first check
to see if the PDF has fillable form fields.
Run this script from this file's directory:
`python scripts/check_fillable_fields <file.pdf>`,
and depending on the result go to either the
"Fillable fields"
or "Non-fillable fields" and follow those
instructions.
```

```
# Fillable fields If the PDF has fillable form
fields:
- Run this script from this file's directory:
`python ./extract_fields.py
<input.pdf> <fields.json>`.
...
```

pdf/extract_fields.py

```
from pypdf import PdfReader

def write_field_info(pdf_path: str, output_path: str):
    """Extract form fields from PDF and store as JSON."""
    reader = PdfReader(pdf_path)
    fields = get_fields(reader)
    with open(output_path, "w") as f:
        json.dump(fields, f)

# ... omitted ...

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print(f"Usage: python {sys.argv[0]} <pdf_path> <output_json_path>")
        sys.exit(1)
    write_field_info(sys.argv[1], sys.argv[2])
```

Skills can also include code for Claude to execute as tools at its discretion based on the nature of the task.

Agent Skills - Progressive Disclosure

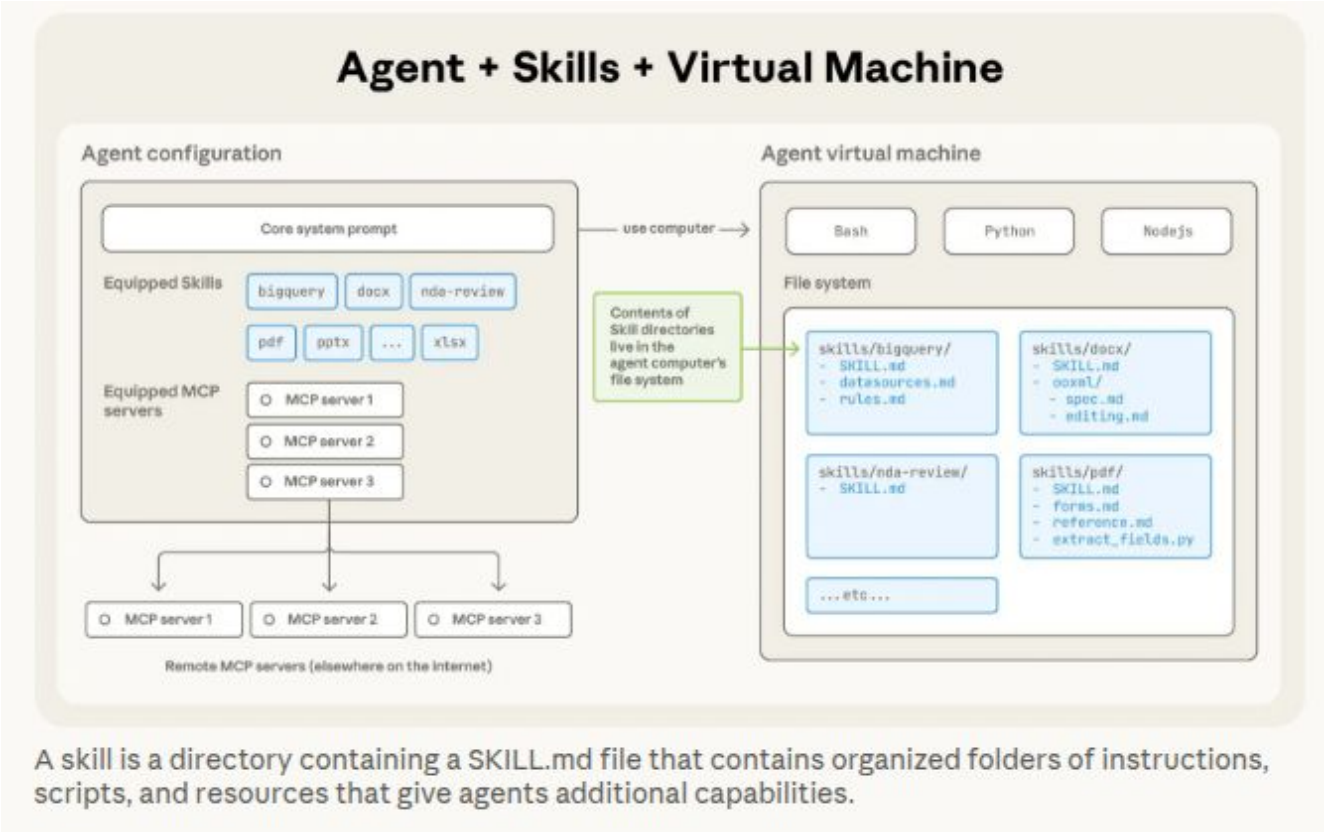
Los agentes cargan las habilidades a través de la revelación progresiva (*progressive disclosure*), en tres etapas:

- **Descubrimiento:** Al inicio, los agentes cargan solo el nombre y la descripción de cada habilidad disponible, lo justo para saber cuándo podría ser relevante.
- **Activación:** Cuando una tarea coincide con la descripción de una habilidad, el agente lee las instrucciones completas del archivo **SKILL.md** para incluirlas en su contexto.
- **Ejecución:** El agente sigue las instrucciones, ejecutando opcionalmente el código empaquetado o cargando los archivos de referencia según sea necesario.

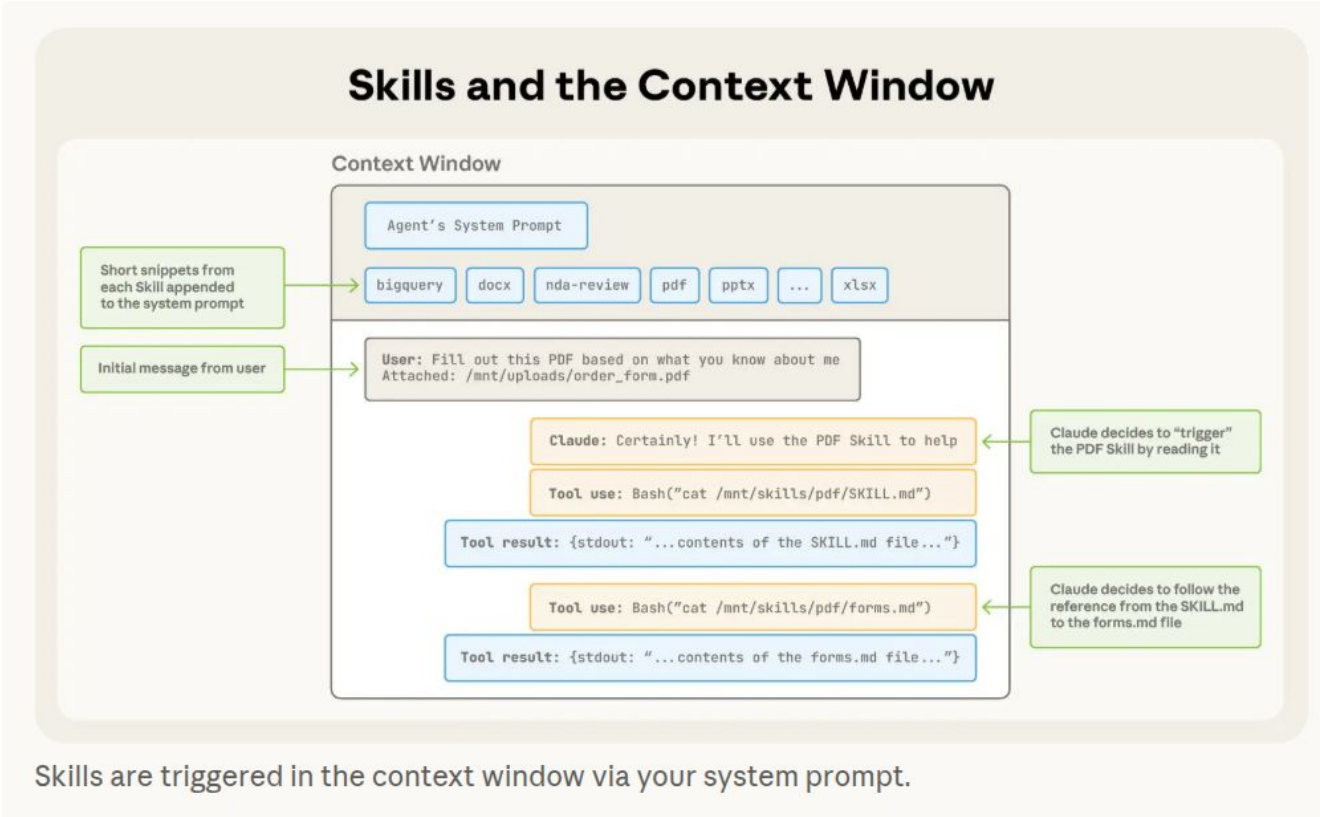
Esta organización permite también reutilizar aquellas cosas que funcionaron.

Level	File	Context Window	# Tokens
1	SKILL.md Metadata (YAML)	Always loaded	~100
2	SKILL.md Body (Markdown)	Loaded when Skill triggers	<5k
3+	Bundled files (text files, scripts, data)	Loaded as-needed by Claude	unlimited*

Agent Skills - Cómo se ejecutan



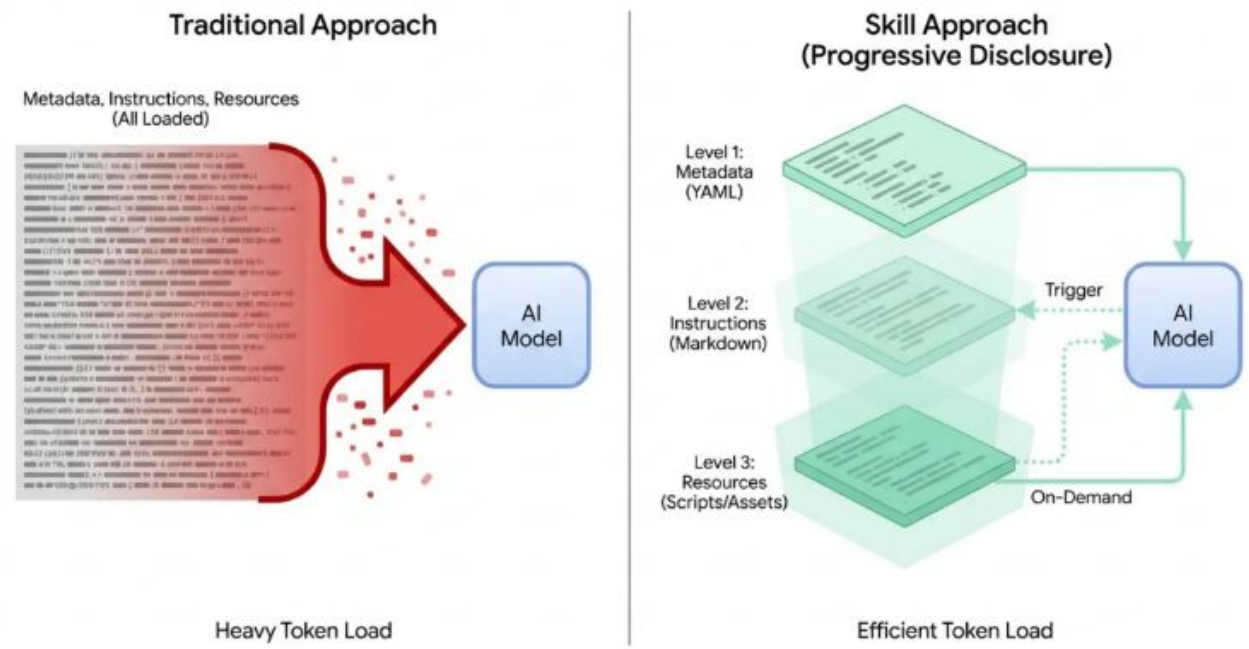
Agent Skills - Cómo se ejecutan



Agent Skills

La revelación progresiva es la forma que se encontró de poder usar distintas habilidades sin incurrir en una saturación del contexto o tool bloat.

Architecture of Efficiency: Progressive Disclosure vs. Static Context



The Progressive Disclosure model (right) drastically reduces initial token load compared to traditional context loading (left). Level 1 loads only lightweight metadata. Level 2 injects procedural instructions only upon trigger. Level 3 executes scripts and reads assets on-demand, keeping the context window streamlined.

Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality

George Ling¹, Shanshan Zhong², Richard Huang¹
¹Bosch Research, ²Carnegie Mellon University

Abstract

Agent skills extend large language model (LLM) agents with reusable, program-like modules that define triggering conditions, procedural logic, and tool interactions. As these skills proliferate in public marketplaces, it is unclear what types are available, how users adopt them, and what risks they pose. To answer these questions, we conduct a large-scale, data-driven analysis of 40,285 publicly listed skills from a major marketplace. Our results show that skill publication tends to occur in short bursts that track shifts in community attention. We also find that skill content is highly concentrated in software engineering workflows, while information retrieval and content creation account for a substantial share of adoption. Beyond content trends, we uncover a pronounced supply-demand imbalance across categories, and we show that most skills remain within typical prompt budgets despite a heavy-tailed length distribution. Finally, we observe strong ecosystem homogeneity, with widespread intent-level redundancy, and we identify non-trivial safety risks, including skills that enable state-changing or system-level actions. Overall, our findings provide a quantitative snapshot of agent skills as an emerging infrastructure layer for agents and inform future work on skill reuse, standardization, and safety-aware design.

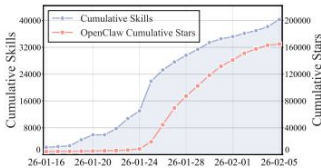


Figure 1: **Growth trend of agent skills.** According to the well-known agent skills platform *skills.sh*, the number of recorded skills experienced rapid growth from mid-January to early February 2026, exceeding 40,000 by early February. During the same period, the popular open-source skills application OpenClaw (*openclaw Community, 2026*) saw a sharp surge in GitHub stars, reaching over 25,000 stars in a single day at the end of January, followed by a gradual decline, with the total number of stars exceeding 170k.

et al., 2023; Sapkota et al., 2025). This agentic execution model has enabled a growing range of applications, from software development and data analysis to personal assistance and workflow automation (Wang et al., 2024).

As agent-based systems scale in both complexity and deployment, new challenges arise around reliability, reuse, and maintainability (Liu et al., 2026a; Cemri et al., 2025; Raheem and Hossain, 2025). Many agent behaviors recur across tasks and users, yet are repeatedly re-specified through

Las habilidades (*skills*) son procedimientos ejecutables que interactúan con servicios externos, entornos locales y el contexto del usuario. En comparación con las interacciones que dependen únicamente de *prompts*, estas expanden la superficie de riesgo (o superficie de daño) porque una habilidad puede acceder a datos sensibles o desencadenar efectos secundarios en el mundo real.

Demo

Usando Skills

en Coding Agentes

(claude code, antigraVity)

Agent Skills - Mejores prácticas, Evaluación y Optimización

Mejores prácticas: <https://agentskills.io/skill-creation/best-practices>

Optimización de Skills: <https://agentskills.io/skill-creation/optimizing-descriptions>

Evaluación de Skills: <https://agentskills.io/skill-creation/evaluating-skills>

Ejemplos de Skills: <https://github.com/rominirani/antigravity-skills>

Tutorial: <https://platform.claude.com/cookbook/skills-notebooks-01-skills-introduction>

Usar skills pre-made anthropic: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/quickstart>

The Complete Guide to Building Skills for Claude: <https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf>

Patrones para Skills ADK: <https://lavinigam.com/posts/adk-skill-design-patterns/>

NPX Skills: <https://github.com/vercel-labs/skills>

Coding in Claude Code: <https://code.claude.com/docs/en/features-overview#subagent-vs-agent-team>

Custom subagents: <https://code.claude.com/docs/en/sub-agents>

Claude Agents: <https://code.claude.com/docs/en/agents>

Cookbooks Claude: https://github.com/anthropics/claude-cookbooks/blob/main/managed_agents/CMA_plan_big_execute_small.ipynb

Uso Responsable de la IA, Gobernanza y Seguridad en Agentes

IA Responsable

- Hace tiempo, los investigadores vienen debatiendo sobre la Ética de la AI, lo que llamaron últimamente como **Inteligencia Artificial Responsable**.
- En esta intersección entre la Ingeniería y el AI, muchas veces se pierde dimensión del cuándo, cómo y dónde debemos usar estos algoritmos de AI siendo **responsables sobre el impacto social y humano**
- Tradicionalmente se definen cinco dimensiones en las cuales se puede evaluar estos sistemas de AI:
 - **Equidad (Fairness)**
 - **Responsabilidad (Accountable)**
 - **Seguridad y Confiabilidad (Secure and Safe)**
 - **Explicabilidad y Transparencia**
 - **Privacidad**
- Es necesario durante la definición del proyecto de Machine Learning, **hay que evaluar la propuesta y diseño del sistema en AI Responsable.** Plantearse posibles escenarios donde dichos supuestos pueden no cumplirse y realizar **mitigaciones**, es decir, **acciones que lleven a anular o mitigar daños.**

IA Responsable - Explicabilidad

*La Explicabilidad se refiere a la capacidad de un modelo de Machine Learning de proporcionar o ser acompañado por detalles o razones que hagan que su funcionamiento sea claro o fácil de entender **para una audiencia específica**."*

Conceptos Relacionados (El triángulo de XAI): Para entender la explicabilidad dos términos que suelen confundirse:

- **Interpretabilidad (Interpretability):** Es la capacidad de explicar o proporcionar el significado en términos comprensibles para un humano. Es una propiedad pasiva: el modelo es interpretable si el humano puede entender su estructura matemática (como un árbol de decisión).
- **Transparencia (Transparency):** Se refiere a que el modelo no sea una "caja negra" (Black-Box). Un modelo es transparente si podemos entender directamente los mecanismos por los cuales funciona, tanto a nivel del algoritmo, como de sus partes y del modelo completo.

<https://arxiv.org/abs/1910.10045>

Evaluating Concept-based Explanations of MLLMs as ICL Visual Classifiers.
ICML 2026. 2nd CompLearn Workshop
<https://arxiv.org/abs/2605.28215>

Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI

Alejandro Barredo Arrieta^a, Natalia Díaz-Rodríguez^b, Javier Del Ser^{a,c,d}, Adrien Bénéto^{b,e,f},
Siham Tabik^g, Alberto Barbado^h, Salvador García^g, Sergio Gil-Lopez^a, Daniel Molina^g,
Richard Benjamins^h, Raja Chatila^d, and Francisco Herrera^g

^aTECNALIA, 48160 Derio, Spain
^bENSTA, Institute Polytechnique Paris and INRIA Flowers Team, Palaiseau, France
^cUniversity of the Basque Country (UPV/EHU), 48013 Bilbao, Spain
^dBasque Center for Applied Mathematics (BCAM), 48009 Bilbao, Bizkaia, Spain
^eSegula Technologies, Parc d'activité de Pissaloup, Trappes, France
^fInstitut des Systèmes Intelligents et de Robotique, Sorbonne Université, France
^gDaSCI Andalusian Institute of Data Science and Computational Intelligence, University of Granada, 18071 Granada, Spain
^hTelefonica, 28050 Madrid, Spain

Abstract

In the last few years, Artificial Intelligence (AI) has achieved a notable momentum that, if harnessed appropriately, may deliver the best of expectations over many application sectors across the field. For this to occur shortly in Machine Learning, the entire community stands in front of the barrier of explainability, an inherent problem of the latest techniques brought by sub-symbolism (e.g. ensembles or Deep Neural Networks) that were not present in the last hype of AI (namely, expert systems and rule based models). Paradigms underlying this problem fall within the so-called *eXplainable* AI (XAI) field, which is widely acknowledged as a crucial feature for the practical deployment of AI models. The overview presented in this article examines the existing literature and contributions already done in the field of XAI, including a prospect toward what is yet to be reached. For this purpose we summarize previous efforts made to define explainability in Machine Learning, establishing a novel definition of explainable Machine Learning that covers such prior conceptual propositions with a major focus on the audience for which the explainability is sought. Departing from this definition, we propose and discuss about a taxonomy of recent contributions related to the explainability of different Machine Learning models, including those aimed at explaining Deep Learning methods for which a second dedicated taxonomy is built and examined in detail. This critical literature analysis serves as the motivating background for a series of challenges faced by XAI, such as the interesting crossroads of data fusion and explainability. Our prospects lead toward the concept of *Responsible Artificial Intelligence*, namely, a methodology for the large-scale implementation of AI methods in real organizations with fairness, model explainability and accountability at its core. Our ultimate goal is to provide newcomers to the field of XAI with a thorough taxonomy that can serve as reference material in order to stimulate future research advances, but also to encourage experts and professionals from other disciplines to embrace the benefits of AI in their activity sectors, without any prior bias for its lack of interpretability.

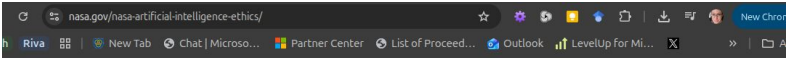
Keywords: Explainable Artificial Intelligence, Machine Learning, Deep Learning, Data Fusion, Interpretability, Comprehensibility, Transparency, Privacy, Fairness, Accountability, Responsible Artificial Intelligence.

arXiv:1910.10045v2 [cs.AI] 26 Dec 2019

Privacidad de la Información Personal

- Hay consideraciones importantes en cuento a la Privacidad y Seguridad de nuestros valores éticos en Inteligencia Artificial:
 - **Propiedad de los datos** - Es importante tener en cuenta quien es el propietario de los datos, ya sea del dataset como de la información personal confiada a las empresas o laboratorio.
 - **Manipulación de los datos** - Trabajar con datos personales o propietarios conlleva una responsabilidad. Así los datos de producción, son cuidados y su acceso es restringido, contar con dichos accesos no significa estar habilitados a sacarlos del ambiente o distribuirlos. En cualquier caso, tener en cuenta situaciones en que los datos deben ser preprocesados para eliminar información:
 - personal (PII - Personal Identifiable Information)
 - identificable médica (PHI - Personal Health Information)

Política de IA y Gobernanza

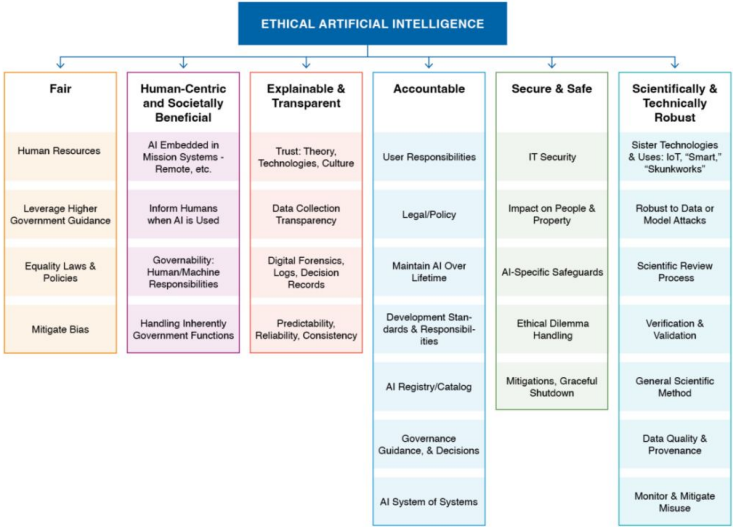


Using AI in a trustworthy and ethical manner is important to us.

While use of AI continues to develop, NASA understands that usage must come with governance and protections for the benefit of all.

NASA Framework for the Ethical Use of Artificial Intelligence

The NASA Framework for the Ethical Use of Artificial Intelligence (AI) provides six key principles to guide NASA's use of AI. The framework describes each ethical AI principle and applies it to work done at NASA. It also includes a list of questions that practitioners should use to guide their AI work. The framework focuses on concrete, practical considerations for the next five to ten years, while also beginning to lay the foundation for longer-term disruptive change as human-level (or beyond) AI is developed.



La norma **ISO/IEC 42001** (publicada originalmente a finales de 2023 y consolidada como el estándar de oro global) es el primer estándar internacional del mundo para implementar un **Sistema de Gestión de Inteligencia Artificial (SGIA)**.

La ISO 42001 es una norma de certificación (al estilo de la ISO 9001 de calidad o la ISO 27001 de seguridad de la información). Establece de forma muy estructurada **cómo debe ser una Política de IA dentro de una organización**.

IA justa e inclusiva

Promovemos la precisión de los resultados que producen nuestros sistemas de IA para tomar decisiones justas y confiables. Trabajamos, además, para garantizar que nuestra IA sea representativa y accesible, y para fomentar que las aplicaciones de tecnología de IA produzcan resultados justos, promuevan la inclusión y garanticen la equidad, logrando que los beneficios de la IA sean accesibles para cualquier persona.

Esto implica que los resultados no deberían conducir a impactos discriminatorios por razón de género, raza, origen étnico, religión, orientación sexual, discapacidad u otro tipo de condición. Por ello, trabajamos en aplicar la

o está clasificado como PUBLICO por TELEFONICA.
is classified as PUBLIC by TELEFONICA.

tecnología para reducir al mínimo la probabilidad de que los conjuntos de datos de entrenamiento que usemos creen o refuercen un sesgo o una discriminación injusta, así como los que se pueden introducir en el diseño e implementación de los algoritmos. Creemos que la IA puede proporcionar una oportunidad de que las personas puedan tomar decisiones más justas y equilibradas, siempre que adoptemos un enfoque de diseño responsable, justo e inclusivo.

IA respetuosa de la privacidad y la seguridad

Los sistemas de IA se basan en datos, que en muchos casos pueden ser de carácter personal, tanto en las etapas de modelado y entrenamiento de los sistemas, como cuando estos se encuentran en producción. En Telefónica nos comprometemos a respetar el derecho a la protección de datos y a la privacidad de las personas. Al procesar datos personales, y de acuerdo con nuestra política de privacidad, cumpliremos en todo momento con los principios de licitud, lealtad y transparencia, minimización de datos, limitación en su conservación y en su finalidad, exactitud, integridad y confidencialidad, siempre cumpliendo con el principio de responsabilidad activa. Para garantizar el cumplimiento de nuestra política de privacidad, utilizamos una metodología de privacidad desde el diseño.

Política de IA

Dada la naturaleza no determinística de la IA y los riesgos posibles que surgen en las distintas etapas de la creación, uso y actualización de los sistemas de AI, generalmente compañías e instituciones siguen principios éticos y de Responsabilidad en Inteligencia Artificial que los define.

Estos principios suelen ser:

- **Equidad y Mitigación de Sesgos (Fairness):** Se trata de asegurarnos proactivamente de que nuestros algoritmos y datos de entrenamiento no generen resultados injustos o discriminatorios para las personas.
- **Integridad de los Datos y Privacidad (Privacy):** Es nuestro compromiso de usar únicamente los datos exactos y estrictamente necesarios, protegiendo siempre el derecho de los usuarios a acceder, corregir o borrar su información de nuestros sistemas.
- **Transparencia y Explicabilidad:** Consiste en ser súper claros sobre cómo usamos los datos y lograr traducir las decisiones complejas de la IA en razones intuitivas que cualquier persona pueda entender.
- **Diseño Centrado en el Humano:** Significa no dejar nunca a la máquina sola, integrando siempre la supervisión humana a lo largo del ciclo de vida del agente para auditar, ajustar y mantener el control sobre las decisiones.
- **Seguridad (Security & Safety):** Se enfoca en armar una defensa técnica robusta para proteger a nuestros modelos y bases de datos frente a ciberataques modernos, como el envenenamiento de datos o las fugas de información.
- **Sostenibilidad y Cuidado del Ambiente (Environmental Impact):** Nos recuerda que, al evaluar o adquirir nuevas tecnologías de IA, el impacto ecológico y la sostenibilidad son criterios fundamentales que debemos poner sobre la mesa.
- **Responsabilidad y Evaluación de Impacto (Accountability):** Es nuestra obligación, como creadores u operadores, de evaluar los riesgos desde el día uno, implementar salvaguardas y poder demostrar con hechos que estamos cuidando los derechos de todos.

Riesgos de Seguridad

Las fuentes de riesgo incluyen:

- Instrucciones ambiguas para el agente.
- Intentos de inyección de *prompts* y evasión de filtros (*jailbreak*) por parte de usuarios malintencionados.
- Inyecciones de *prompts* indirectas a través del uso de herramientas.

Las categorías de riesgo incluyen:

- **Desalineación y corrupción de objetivos:** Perseguir objetivos no deseados o indirectos que conducen a resultados perjudiciales ("hacking de recompensas").
- **Mala interpretación:** Malinterpretar instrucciones complejas o ambiguas.
- **Generación de contenido perjudicial (incluyendo la seguridad de la marca):** Generar contenido tóxico, de odio, sesgado, sexualmente explícito, discriminatorio o ilegal.
- **Riesgos para la seguridad de la marca:** Como utilizar un lenguaje que vaya en contra de los valores de la marca o tener conversaciones fuera del tema.
- **Acciones inseguras:** Ejecutar comandos que dañen los sistemas.
- **Transacciones no autorizadas:** Realizar compras o transacciones financieras sin autorización.
- **Filtración de datos:** Filtrar datos personales sensibles (PII, por sus siglas en inglés).
- **Exfiltración de datos:** Extracción o robo de datos del sistema.

Principios de Seguridad de Agentes

Para evitar estos riesgos se propone diseñar agentes que:

- **Tengan humanos como controladores:** agregar mecanismos que los usuarios revisen las acciones y la información dando el consentimiento.
- **Que tengan su poder limitado:** darles permisos a los agentes mínimos y controlados para las acciones y sistemas que deban tener acceso por un tiempo definido, auditar las autorizaciones y la infraestructura/servicios a los que acceden. Además, usar credenciales como OAuth tokens de una manera segura.
- **Y lograr que las acciones y los planes del agente sean observables:** La transparencia y observabilidad de las acciones y procesos de planificación de un agente son fundamentales para garantizar la confianza, la auditoría de seguridad y una depuración efectiva. Se requiere invertir en una infraestructura robusta de registros seguros y centralizados que capturen detalladamente las entradas, herramientas utilizadas, salidas y los pasos lógicos intermedios. Las interfaces de usuario y las API deben exponer claramente las características de cada acción (como sus riesgos o manejo de datos sensibles) para que los usuarios puedan comprender y supervisar el "razonamiento" del agente.

Día 3: Temas que vimos...

- Fundamentos teóricos de los agentes de IA, su ciclo iterativo de pensamiento, acción y corrección, y ejemplos de aplicaciones en el mercado como herramientas de programación, chatbots y sistemas IVR.1
- Arquitecturas multiagente, niveles de maduración de los sistemas agénticos y patrones de coordinación complejos.1
- Frameworks de desarrollo destacados como LlamaIndex, LangChain y Google ADK.
- Seguridad en agentes: prevención de inyecciones, medidas de protección y buenas prácticas.1
- Agent Skills: estándares, formato, revelación progresiva y optimización de habilidades.